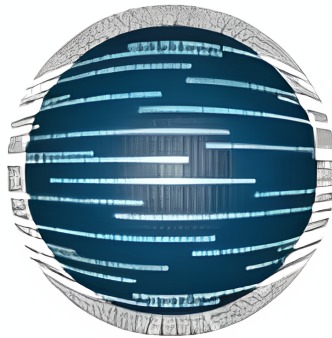




Ταξινόμηση Ανεπιθύμητων Email

Στατιστικές Μέθοδοι Μηχανικής Μάθησης

Βασίλειος Μπίτζας 1083796 up1083796@ac.upatras.gr

Τμήμα Μηχανικών Ηλεκτρονικών Υπολογιστών & Πληροφορικής
Πανεπιστήμιο Πατρών

Περιεχόμενα

1	Εισαγωγή	8
1.1	Εισαγωγή	8
1.2	Στόχοι Εργασίας	8
1.3	Περιγραφή Δεδομένων	9
1.3.1	Dataset: emails.csv	9
1.3.2	Διαχωρισμός Dataset	10
1.3.3	Παράδειγμα Δεδομένων	10
1.4	Διάρθρωση Αναφοράς	11
2	Θεωρητικό Υπόβαθρο	12
2.1	Επεξεργασία Κειμένου	12
2.1.1	Προεπεξεργασία Κειμένου	12
2.1.2	Stopwords	12
2.1.3	Stemming και Lemmatization	13
2.2	Αναπαράσταση Κειμένου	13
2.2.1	Bag of Words (BoW)	13
2.2.2	Word Embeddings	14
2.2.3	Sentence Transformers	15
2.3	Αλγόριθμοι Ταξινόμησης	16
2.3.1	Naive Bayes	16
2.3.2	k-Nearest Neighbors (k-NN)	17
2.3.3	Support Vector Machines (SVM)	19
2.3.4	Logistic Regression	20
2.4	Μείωση Διαστάσεων	22
2.4.1	Principal Component Analysis (PCA)	22
2.4.2	Explained Variance	23
2.5	Μετρικές Αξιολόγησης	24
2.5.1	Confusion Matrix	24
2.5.2	Precision, Recall, F1-Score	24
2.5.3	ROC Curve και AUC	25
3	Μεθοδολογία	26
3.1	Συλλογή και Περιγραφή Δεδομένων	26
3.1.1	Dataset Emails.csv	26
3.1.2	Κατανομή Κλάσεων	26
3.1.3	Exploratory Data Analysis	27
3.2	Διαχωρισμός Δεδομένων	27
3.2.1	Training, Validation, Test Sets	27
3.2.2	Stratified Split	28
3.3	Προεπεξεργασία Κειμένου	29
3.3.1	Πέντε-βημάτων-στάδιο Καθαρισμού	29
3.3.2	Μαθηματική Διατύπωση	30
3.3.3	Απόφαση: Όχι Stemming/Lemmatization	30

3.3.4	Επίδραση Προεπεξεργασίας	31
3.4	Αναπαράσταση Χαρακτηριστικών	31
3.4.1	Bag of Words με CountVectorizer	31
3.4.2	Sentence Embeddings με Transformers	32
3.4.3	Μείωση Διαστάσεων με PCA	34
3.4.4	Σύνοψη Feature Representations	35
3.5	Εκπαίδευση Μοντέλων	35
3.5.1	Baseline Models	36
3.5.2	Συνδυασμοί Features - Models	38
3.5.3	Hyperparameter Tuning	39
3.5.4	Validation Strategy	40
3.5.5	Evaluation Metrics	40
3.5.6	Implementation Details	41
3.5.7	Computational Considerations	41
4	Υλοποίηση	42
4.1	Περιβάλλον Ανάπτυξης	42
4.1.1	Τεχνολογίες και Εργαλεία	42
4.1.2	Βιβλιοθήκες Μηχανικής Μάθησης	42
4.2	Αρχιτεκτονική Κώδικα	43
4.2.1	Δομή Notebook	43
4.2.2	Κύριες Συναρτήσεις	43
4.3	Προεπεξεργασία	44
4.3.1	Υλοποίηση Καθαρισμού Κειμένου	44
4.3.2	Feature Extraction Pipeline	45
4.4	Εκπαίδευση Μοντέλων	46
4.4.1	Naive Bayes Implementation	46
4.4.2	k-NN Implementation	46
4.4.3	Logistic Regression Implementation	47
4.4.4	SVM Implementation	48
4.4.5	PCA Implementation	48
4.5	Αξιολόγηση	49
4.5.1	Evaluation Pipeline	49
4.5.2	Οπτικοποιήσεις	50
5	Αποτελέσματα	51
5.1	Συγκριτικός Πίνακας Αποτελεσμάτων	51
5.1.1	Στατιστική Ανάλυση	51
5.2	Naive Bayes με Bag-of-Words	52
5.2.1	Απόδοση στο Test Set	52
5.2.2	Confusion Matrix	52
5.3	Logistic Regression με Sentence Embeddings	54
5.3.1	Απόδοση στο Test Set	54
5.3.2	Confusion Matrix	54
5.4	k-Nearest Neighbors	55
5.4.1	Επιλογή Βέλτιστου k	55
5.4.2	Τελική Απόδοση (k=1)	56
5.4.3	Confusion Matrix	56
5.5	Support Vector Machine (SVM)	57

5.5.1	Απόδοση στο Test Set	57
5.5.2	Confusion Matrix	57
5.5.3	Kernel Comparison	58
5.6	Επίδραση PCA	58
5.6.1	SVM με PCA (90% Variance)	58
5.6.2	Logistic Regression με PCA (10 components)	60
5.6.3	PCA Variance Ratios	61
5.7	Σύγκριση Precision vs Recall	62
5.8	Κατάταξη Μοντέλων	63
6	Συζήτηση	65
6.1	Ανάλυση Αποτελεσμάτων	65
6.1.1	Bag-of-Words vs Sentence Embeddings	65
6.1.2	Επίδραση των Hyperparameters	68
6.1.3	Απόδοση στα Διαφορετικά Σύνολα Δεδομένων	72
6.2	Περιορισμοί της Μελέτης	73
6.2.1	Περιορισμοί Dataset	73
6.2.2	Περιορισμοί Μοντέλων	74
6.3	Πρακτικές Εφαρμογές	74
6.3.1	Επιλογή Μοντέλου για Production	74
6.3.2	Deployment Architecture	75
7	Συμπεράσματα	76
7.1	Κύρια Ευρήματα	76
7.1.1	Βασικά Συμπεράσματα	76
7.1.2	Συνεισφορά της Εργασίας	77
7.2	Μελλοντικές Κατευθύνσεις	77
7.2.1	Βελτιώσεις Μοντέλων	77
7.2.2	Επέκταση Dataset	78
7.2.3	Ερευνητικές Ερωτήσεις	78
7.2.4	Κοινωνικές Επιπτώσεις	78
7.3	Τελικό Συμπέρασμα	78
A'	Πλήρης Πίνακας Μοντέλων	80
A'.1	Αναλυτικά Αποτελέσματα	80
A'.1.1	Training και Inference Times	80
A'.1.2	Confusion Matrices Σύνοψη	81
B'	Επιλεγμένος Κώδικας	82
B'.1	Προεπεξεργασία Κειμένου	82
B'.2	Feature Extraction	83
B'.2.1	Bag-of-Words με TfidfVectorizer	83
B'.2.2	Sentence Embeddings με SBERT	83
B'.3	Εκπαίδευση Μοντέλων	84
B'.3.1	Naive Bayes	84
B'.3.2	Support Vector Machine	84
B'.3.3	Dimensionality Reduction με PCA	85
B'.4	Αξιολόγηση και Visualizations	85
B'.4.1	Confusion Matrix και Metrics	85

Β'.4.2	ROC Curve Comparison	86
Γ	Οδηγίες Εγκατάστασης και Εκτέλεσης	87
Γ'.1	Προαπαιτούμενα	87
Γ'.1.1	Απαιτούμενο Λογισμικό	87
Γ'.1.2	Python Libraries	87
Γ'.1.3	Απαιτήσεις Συστήματος	88
Γ'.2	Εγκατάσταση	88
Γ'.2.1	Βήμα 1: Clone το Repository	88
Γ'.3	Εκτέλεση του Notebook	88
Γ'.3.1	Έναρξη Jupyter	88
Γ'.3.2	Άνοιγμα του Notebook	88
Γ'.3.3	Εκτέλεση Cells	89
Γ'.3.4	Troubleshooting	89
Γ'.3.5	Επιπλέον Πόροι	89

Κατάλογος Σχημάτων

5.1	Naive Bayes: Σύγκριση απόδοσης validation vs test set και ROC curve	53
5.2	ROC Curve για Naive Bayes (AUC=0.9959)	53
5.3	Ανάλυση variance components για PCA - Cumulative variance ratio	61
5.4	Σύγκριση original embeddings vs διαφορετικά PCA configurations	61
5.5	Σύγκριση βασικών μοντέλων: Naive Bayes, Logistic Regression, k-NN, SVM	63
5.6	F1-Score comparison για τα βασικά μοντέλα	63
5.7	Συνολική σύγκριση όλων των μοντέλων: Heatmap, Radar chart, Bar charts, και τελική κατάταξη	64
5.8	Σύγκριση απόδοσης όλων των μοντέλων με όλα τα metrics	64
6.1	Σύγκριση απόδοσης Bag-of-Words vs Sentence Embeddings	67
6.2	ROC Curves: BoW (Naive Bayes) vs Embeddings (SVM, LogReg)	67
6.3	Confusion matrices: Σύγκριση BoW vs Embeddings στο test set	68
6.4	Top 15 λέξεις που χαρακτηρίζουν Ham vs Spam emails	68
6.5	k-NN: Επίδραση του k στην απόδοση και bias-variance trade-off	69
6.6	SVM: Σύγκριση kernels (Linear, Polynomial, RBF) στο validation set	70
6.7	SVM Polynomial kernel: Confusion matrix, ROC curves, και σύγκριση kernels	71
6.8	Logistic Regression με PCA(10): ROC curve και confusion matrices	72
6.9	Σύγκριση confusion matrices: Ποσοστά και απόλυτοι αριθμοί για όλα τα σύνολα δεδομένων	74

Κατάλογος Πινάκων

3.1	Κατανομή κλάσεων στο dataset emails.csv	26
3.2	Διαχωρισμός δεδομένων σε Training, Validation και Test sets	27
3.3	Συνολικές αναπαραστάσεις χαρακτηριστικών που δοκιμάστηκαν	35
3.4	Συνδυασμοί μοντέλων και feature representations που δοκιμάστηκαν	38
3.5	PCA configurations με variance-based component selection από 384 αρχικές διαστάσεις	40
5.1	Συγκριτικά αποτελέσματα όλων των μοντέλων στο test set	51
5.2	Στατιστικά στοιχεία απόδοσης μοντέλων	51
5.3	Naive Bayes αποτελέσματα	52
5.4	Confusion Matrix για Naive Bayes	52
5.5	Logistic Regression με Sentence Embeddings	54
5.6	Confusion Matrix για Logistic Regression	54
5.7	Σύγκριση k-NN για διαφορετικές τιμές k (validation set)	55
5.8	k-NN ($k = 1$) αποτελέσματα στο test set	56
5.9	Confusion Matrix για k-NN ($k = 1$)	56
5.10	SVM με Polynomial kernel - Υψηλότερο F1-Score και Υψηλότερο AUC	57
5.11	Confusion Matrix για SVM (Polynomial)	57
5.12	Σύγκριση SVM με και χωρίς PCA	58
5.13	Confusion Matrix για SVM με PCA (90% variance, 115 dimensions)	59
5.14	Σύγκριση Logistic Regression με δραστική μείωση διαστάσεων	60
5.15	Confusion Matrix για Logistic Regression με PCA (10 components)	60
5.16	PCA configurations και μείωση διαστάσεων	61
5.17	Precision και Recall ανά μοντέλο	62
6.1	Σύγκριση BoW vs Sentence Embeddings	66
6.2	Επίδραση του k στην απόδοση k-NN (validation set)	69
6.3	Σύγκριση SVM kernels (test set)	69
6.4	Trade-off μεταξύ variance retention και απόδοσης	71
6.5	Validation vs Test set performance	72
6.6	Κατανομή κλάσεων στα σύνολα δεδομένων	73
6.7	Υπολογιστικές απαιτήσεις μοντέλων	74
A.1	Πλήρης πίνακας αποτελεσμάτων για όλα τα μοντέλα	80
A.2	Υπολογιστικές απαιτήσεις κάθε μοντέλου	80
A.3	Confusion matrices για όλα τα μοντέλα στο test set	81

Κεφάλαιο 1

Εισαγωγή

1.1 Εισαγωγή

Το ηλεκτρονικό ταχυδρομείο (email) αποτελεί έναν από τους πιο διαδεδομένους τρόπους επικοινωνίας στον ψηφιακό κόσμο, με περισσότερα από 4 δισεκατομμύρια χρήστες παγκοσμίως. Ωστόσο, η ευρεία χρήση του email έχει οδηγήσει σε ένα σοβαρό πρόβλημα: το **spam** (ανεπιθύμητα μηνύματα). Σύμφωνα με στατιστικά στοιχεία, περίπου το 45% έως 50% όλων των email είναι spam, προκαλώντας σημαντικά προβλήματα στους χρήστες και τις επιχειρήσεις.

Το spam δεν είναι απλώς μια ενόχληση. Περιλαμβάνει διαφημιστικά μηνύματα, απόπειρες ηλεκτρονικού ψαρέματος (phishing), κακόβουλο λογισμικό (malware), και άλλες απειλές που μπορούν να θέσουν σε κίνδυνο την ασφάλεια και την ιδιωτικότητα των χρηστών. Επιπλέον, το spam καταναλώνει σημαντικούς πόρους δικτύου και αποθηκευτικού χώρου, ενώ μειώνει την παραγωγικότητα των χρηστών που αναγκάζονται να ξοδεύουν χρόνο φιλτράροντας ανεπιθύμητα μηνύματα.

Η **αυτόματη ανίχνευση spam** (spam detection) έχει γίνει κρίσιμη ανάγκη. Οι παραδοσιακές μέθοδοι βασισμένες σε κανόνες (rule-based systems) δεν μπορούν να ανταπεξέλθουν στην εξέλιξη των τεχνικών spam, καθώς οι αποστολές spam προσαρμόζονται συνεχώς για να παρακάμψουν τα φίλτρα. Αντίθετα, οι **τεχνικές μηχανικής μάθησης** (machine learning) προσφέρουν μια πιο ευέλικτη και προσαρμοστική προσέγγιση, μαθαίνοντας αυτόματα από παραδείγματα και βελτιώνοντας συνεχώς την απόδοσή τους.

Η παρούσα εργασία επικεντρώνεται στην **ταξινόμηση email** ως spam ή ham (legitimate email) χρησιμοποιώντας σύγχρονες τεχνικές μηχανικής μάθησης και επεξεργασίας φυσικής γλώσσας (Natural Language Processing - NLP). Συγκεκριμένα, εξετάζουμε διαφορετικές μεθόδους αναπαράστασης κειμένου (Bag-of-Words και Sentence Embeddings) και εφαρμόζουμε διάφορους αλγορίθμους ταξινόμησης (Naive Bayes, k-Nearest Neighbors, Support Vector Machines, Logistic Regression) για να αξιολογήσουμε την αποτελεσματικότητά τους.

1.2 Στόχοι Εργασίας

Η παρούσα εργασία έχει τους ακόλουθους βασικούς στόχους:

1. Υλοποίηση Pipeline Προεπεξεργασίας Κειμένου

- Καθαρισμός κειμένου (αφαίρεση URLs, ειδικών χαρακτήρων)
- Κανονικοποίηση (μετατροπή σε πεζά, κανονικοποίηση κενών)
- Αφαίρεση stopwords από το κείμενο των email
- Αξιολόγηση της επίδρασης της προεπεξεργασίας στην απόδοση

2. Σύγκριση Μεθόδων Αναπαράστασης Κειμένου

- Bag-of-Words (BoW):** Παραδοσιακή προσέγγιση βασισμένη σε συχνότητες λέξεων
- Sentence Embeddings:** Σύγχρονη προσέγγιση χρησιμοποιώντας pre-trained transformers

(all-MiniLM-L6-v2)

- Αξιολόγηση της σχέσης απόδοσης/πολυπλοκότητας για κάθε μέθοδο

3. Εφαρμογή και Σύγκριση Αλγορίθμων Ταξινόμησης

- **Naive Bayes:** Πιθανοτικός ταξινομητής για baseline comparison
- **k-Nearest Neighbors (k-NN):** Instance-based learning
- **Support Vector Machines (SVM):** Με linear και RBF kernels
- **Logistic Regression:** Γραμμικό μοντέλο με L2 regularization
- Αξιολόγηση με πολλαπλές μετρικές (Precision, Recall, F1-Score, ROC-AUC)

4. Μελέτη Επίδρασης Μείωσης Διαστάσεων

- Εφαρμογή Principal Component Analysis (PCA) στα sentence embeddings
- Δοκιμή διαφορετικών variance ratios (90%, 95%, 99%) και σταθερών components (10)
- Ανάλυση του trade-off μεταξύ μείωσης διαστάσεων και απώλειας απόδοσης

5. Αξιολόγηση και Σύγκριση Μοντέλων

- Συγκριτική ανάλυση όλων των συνδυασμών (αναπαραστάσεις × αλγόριθμοι × PCA)
- Ανάλυση σφαλμάτων (confusion matrices, ROC curves)
- Προσδιορισμός του βέλτιστου μοντέλου για διαφορετικά use cases

6. Πρακτικές Συστάσεις

- Προτάσεις για deployment σε production περιβάλλον
- Ανάλυση υπολογιστικής πολυπλοκότητας και scalability
- Προτάσεις για μελλοντικές βελτιώσεις

Η εργασία στοχεύει στην παροχή μιας ολοκληρωμένης ανάλυσης των σύγχρονων τεχνικών για την ανίχνευση spam, παρέχοντας τόσο θεωρητική κατανόηση όσο και πρακτική εφαρμογή.

1.3 Περιγραφή Δεδομένων

1.3.1 Dataset: emails.csv

Για την υλοποίηση της εργασίας χρησιμοποιείται το dataset *emails.csv*, το οποίο περιέχει συλλογή email με τα ακόλουθα χαρακτηριστικά:

- **Μέγεθος Dataset:** Σύνολο 5,728 emails
- **Στήλες:**
 - *text*: Το περιεχόμενο του email (string)
 - *spam*: Η ετικέτα κλάσης (binary: 0 = ham, 1 = spam)
- **Κατανομή Κλάσεων:**
 - Ham (legitimate emails): 4,360 δείγματα (76.1%)
 - Spam (unwanted emails): 1,368 δείγματα (23.9%)
- **Ποικιλία Περιεχομένου:** Τα email περιλαμβάνουν διάφορους τύπους spam:
 - Διαφημιστικά μηνύματα (product promotions)

- Οικονομικές απάτες (financial scams)
- Προσφορές φαρμάκων (pharmaceutical spam)
- Phishing attempts
- **Πραγματικά Δεδομένα:** Το dataset προέρχεται από πραγματικά email, διατηρώντας τη φυσική ποικιλομορφία και πολυπλοκότητα της γλώσσας.

1.3.2 Διαχωρισμός Dataset

Το dataset χωρίζεται σε τρία υποσύνολα για την εκπαίδευση και αξιολόγηση των μοντέλων:

- **Training Set:** 4,582 emails (80% του συνόλου)
 - Χρησιμοποιείται για την εκπαίδευση των μοντέλων
 - Ham: 3,468 emails (75.69%)
 - Spam: 1,114 emails (24.31%)
- **Validation Set:** 572 emails (10% του συνόλου)
 - Χρησιμοποιείται για την επιλογή hyperparameters
 - Ham: 446 emails (77.97%)
 - Spam: 126 emails (22.03%)
- **Test Set:** 574 emails (10% του συνόλου)
 - Χρησιμοποιείται για την τελική αξιολόγηση
 - Ham: 446 emails (77.70%)
 - Spam: 128 emails (22.30%)

Σημείωση: Πριν τον διαχωρισμό, το dataset ανακατεύεται τυχαία (shuffle) με *random_state=42* για αναπαραγωγικότητα των αποτελεσμάτων και για να διασφαλιστεί ότι κάθε υποσύνολο περιέχει αντιπροσωπευτική κατανομή των κλάσεων.

1.3.3 Παράδειγμα Δεδομένων

Τα email στο dataset περιέχουν το θέμα (subject) και το σώμα του μηνύματος. Παράδειγμα spam email:

Παράδειγμα Spam Email

Subject: vaal medz how t pestilent o save on your medlcations over 70% . phar anthropoid mshop - suc mucilage cessfull and proven way to save your mo impendence ney . gauleiter v a jauntily g flection al l woodbind u natural l metcast rac selachian l i...

Παράδειγμα ham (legitimate) email:

Παράδειγμα Ham Email

Subject: re : hello vince thank you for the update . i will be back in the office january 8 . have a great new year gerry - - - - original message - - - - from : vince . j . kaminski @ enron . com [mailto : vince . j . kaminski @ enron . com] sent : thursday , december 21 , 2000 5 : 35 pm to : gsheble @ iastate . edu cc : vince...

1.4 Διάρθρωση Αναφοράς

Η αναφορά οργανώνεται ως εξής:

- **Κεφάλαιο 2:** Παρουσιάζεται το θεωρητικό υπόβαθρο των αλγορίθμων μηχανικής μάθησης που χρησιμοποιούνται.
- **Κεφάλαιο 3:** Περιγράφεται η μεθοδολογία που ακολουθήθηκε για την επεξεργασία δεδομένων και την εκπαίδευση μοντέλων.
- **Κεφάλαιο 4:** Παρουσιάζονται λεπτομέρειες υλοποίησης και τεχνικές επιλογές.
- **Κεφάλαιο 5:** Αναλύονται τα αποτελέσματα των μοντέλων που εκπαιδεύτηκαν.
- **Κεφάλαιο 6:** Συζητούνται τα ευρήματα, οι περιορισμοί και οι πρακτικές εφαρμογές.
- **Κεφάλαιο 7:** Παρουσιάζονται τα συμπεράσματα και προτάσεις για μελλοντική έρευνα.

Κεφάλαιο 2

Θεωρητικό Υπόβαθρο

2.1 Επεξεργασία Κειμένου

Η επεξεργασία κειμένου (text processing) αποτελεί το θεμέλιο της ανάλυσης φυσικής γλώσσας και της κατηγοριοποίησης κειμένων. Περιλαμβάνει τη μετατροπή του ακατέργαστου κειμένου σε μια μορφή κατάλληλη για αλγορίθμους μηχανικής μάθησης.

2.1.1 Προεπεξεργασία Κειμένου

Η προεπεξεργασία κειμένου είναι το πρώτο και κρίσιμο βήμα στην ανάλυση κειμένου. Στόχος της είναι η αφαίρεση του θορύβου και η κανονικοποίηση του κειμένου για να βελτιώσει την απόδοση των μοντέλων.

Βασικές Τεχνικές

- Αφαίρεση URLs και Ειδικών Χαρακτήρων:** Τα emails συχνά περιέχουν URLs, διευθύνσεις email, και ειδικούς χαρακτήρες που δεν συνεισφέρουν στην κατηγοριοποίηση. Η αφαίρεσή τους γίνεται μέσω regular expressions.
- Μετατροπή σε Πεζά Γράμματα (Lowercasing):** Η μετατροπή όλων των χαρακτήρων σε πεζά εξασφαλίζει ότι οι λέξεις "Spam", "spam" και "SPAM" αντιμετωπίζονται ως η ίδια λέξη, μειώνοντας το μέγεθος του λεξιλογίου.
- Αφαίρεση Στίξης:** Η στίξη (σημεία στίξης, ειδικοί χαρακτήρες) συνήθως αφαιρείται καθώς δεν προσφέρει σημασιολογική πληροφορία για την κατηγοριοποίηση spam/ham.
- Κανονικοποίηση (Normalization):** Η κανονικοποίηση περιλαμβάνει την αφαίρεση διαδοχικών διπλότυπων χαρακτήρων (π.χ., "heeello" → "hello") και την επαναλαμβανόμενη στίξη που συχνά χρησιμοποιείται σε spam emails.
- Tokenization:** Η διαίρεση του κειμένου σε επιμέρους λέξεις (tokens) που θα χρησιμοποιηθούν ως features από τους αλγορίθμους κατηγοριοποίησης.

Μαθηματικά, η διαδικασία προεπεξεργασίας μπορεί να περιγραφεί ως συνάρτηση:

$$f : D \rightarrow D' \quad (2.1)$$

όπου $D = \{d_1, d_2, \dots, d_n\}$ είναι το σύνολο των αρχικών κειμένων και $D' = \{d'_1, d'_2, \dots, d'_n\}$ το σύνολο των προεπεξεργασμένων κειμένων. Για κάθε έγγραφο $d_i \in D$:

$$d'_i = \text{remove_stopwords}(\text{normalize}(\text{clean}(d_i))) \quad (2.2)$$

2.1.2 Stopwords

Τα stopwords είναι συχνά εμφανιζόμενες λέξεις που έχουν μικρή σημασιολογική αξία για την κατηγοριοποίηση. Στην αγγλική γλώσσα περιλαμβάνουν λέξεις όπως "the", "is", "a", "an", "in", "on" κ.λπ.

Η αφαίρεση stopwords βασίζεται σε μια προκαθορισμένη λίστα λέξεων $S = \{s_1, s_2, \dots, s_k\}$. Για ένα έγγραφο με tokens $T = \{t_1, t_2, \dots, t_m\}$, το φιλτραρισμένο σύνολο δίνεται από:

$$T' = T \setminus S = \{t \in T \mid t \notin S\} \quad (2.3)$$

Στην παρούσα εργασία χρησιμοποιείται η λίστα stopwords της βιβλιοθήκης NLTK, η οποία περιέχει 179 αγγλικές λέξεις. Η αφαίρεση stopwords έχει δύο κύρια πλεονεκτήματα:

- **Μείωση διαστατικότητας:** Μειώνει το μέγεθος του λεξιλογίου κατά 20-30%, επιταχύνοντας την εκπαίδευση.
- **Βελτίωση ακρίβειας:** Αφαιρεί λέξεις που δεν συνεισφέρουν στη διάκριση spam/ham, εστιάζοντας σε περισσότερο πληροφοριακές λέξεις.

2.1.3 Stemming και Lemmatization

Το stemming και το lemmatization είναι τεχνικές κανονικοποίησης που μειώνουν τις λέξεις στη βασική τους μορφή.

Stemming

Το stemming αφαιρεί επιθήματα και προθήματα χρησιμοποιώντας ευριστικούς κανόνες. Για παράδειγμα, ο αλγόριθμος Porter Stemmer μετατρέπει:

- "running", "runs", "ran" → "run"
- "happiness", "happy" → "happi"

Μαθηματικά, το stemming μπορεί να θεωρηθεί ως συνάρτηση $stem : V \rightarrow V'$ όπου V είναι το λεξιλόγιο και V' το μειωμένο λεξιλόγιο stems με $|V'| < |V|$.

Lemmatization

Το lemmatization χρησιμοποιεί λεξικολογική ανάλυση για να μειώσει τις λέξεις στο λήμμα τους (τη λεξικογραφική μορφή). Για παράδειγμα:

- "running", "runs", "ran" → "run"
- "better" → "good"
- "am", "are", "is" → "be"

Το lemmatization είναι πιο ακριβές από το stemming αλλά υπολογιστικά πιο ακριβό. Στην παρούσα εργασία, λόγω του μεγέθους του dataset και της ανάγκης για γρήγορη εκπαίδευση, δεν εφαρμόζεται stemming ή lemmatization, καθώς οι σύγχρονες μέθοδοι όπως οι Sentence Transformers μπορούν να αντιμετωπίσουν μορφολογικές παραλλαγές.

2.2 Αναπαράσταση Κειμένου

Η αναπαράσταση κειμένου (text representation) μετατρέπει το κείμενο σε αριθμητική μορφή που μπορούν να επεξεργαστούν οι αλγόριθμοι μηχανικής μάθησης. Οι κύριες προσεγγίσεις διακρίνονται σε παραδοσιακές (sparse representations) και σύγχρονες (dense representations).

2.2.1 Bag of Words (BoW)

Το μοντέλο Bag of Words (BoW) αναπαριστά κάθε έγγραφο ως ένα διάνυσμα συχνοτήτων λέξεων, αγνοώντας τη σειρά και τη γραμματική δομή.

Μαθηματικό Μοντέλο

Έστω $V = \{w_1, w_2, \dots, w_{|V|}\}$ το λεξιλόγιο (vocabulary) όλων των μοναδικών λέξεων στο corpus. Για ένα έγγραφο d , η αναπαράσταση BoW είναι ένα διάνυσμα:

$$\mathbf{x}_d = [x_1, x_2, \dots, x_{|V|}] \quad (2.4)$$

όπου x_i είναι η συχνότητα εμφάνισης της λέξης w_i στο έγγραφο d :

$$x_i = \text{count}(w_i, d) \quad (2.5)$$

Παράδειγμα

Για τα έγγραφα:

1. "John likes to watch movies. Mary likes movies too."
2. "Mary also likes to watch football games."

Το λεξιλόγιο είναι: $V = \{"john", "likes", "to", "watch", "movies", "mary", "too", "also", "football", "games"\}$

Οι αναπαραστάσεις BoW είναι:

$$\text{Doc}_1 = [1, 2, 1, 1, 2, 1, 1, 0, 0, 0] \quad (2.6)$$

$$\text{Doc}_2 = [0, 1, 1, 1, 0, 1, 0, 1, 1, 1] \quad (2.7)$$

Υλοποίηση με CountVectorizer

Στην πράξη, χρησιμοποιείται το *CountVectorizer* του scikit-learn:

- **max_features**: Περιορίζει το λεξιλόγιο στις k πιο συχνές λέξεις (στην εργασία: 5000)
- **Term frequency**: Χρησιμοποιεί τον αριθμό εμφανίσεων κάθε λέξης (counts)

Η μείωση διαστάσεων από $|V|$ σε k features βελτιώνει την υπολογιστική αποδοτικότητα και μειώνει την υπερεκπαίδευση.

2.2.2 Word Embeddings

Τα word embeddings αναπαριστούν λέξεις ως πυκνά διανύσματα πραγματικών αριθμών σε χαμηλής διάστασης χώρο, όπου σημασιολογικά παρόμοιες λέξεις έχουν παρόμοια διανύσματα.

Word2Vec

Το Word2Vec χρησιμοποιεί νευρωνικά δίκτυα για να μάθει αναπαραστάσεις λέξεων από μεγάλα corpora. Δύο αρχιτεκτονικές:

- **CBOW (Continuous Bag of Words)**: Προβλέπει μια λέξη από το περιβάλλον της
- **Skip-gram**: Προβλέπει το περιβάλλον από μια λέξη

GloVe

Το GloVe (Global Vectors) μαθαίνει embeddings εκμεταλλευόμενο τη μήτρα συν-εμφάνισης λέξεων σε όλο το corpus.

Περιορισμοί

Τα word embeddings έχουν περιορισμούς για την κατηγοριοποίηση εγγράφων:

- Χρειάζεται aggregation (π.χ., μέσος όρος) για αναπαράσταση προτάσεων
- Δεν αιχμαλωτίζουν πλήρως τη σύνθεση φράσεων
- Απαιτούν μεγάλα δεδομένα εκπαίδευσης

2.2.3 Sentence Transformers

Οι Sentence Transformers χρησιμοποιούν μετασχηματιστές (transformers) για να δημιουργήσουν σταθερής διάστασης embeddings για ολόκληρες προτάσεις/εγγράφα.

Μαθηματική Αναπαράσταση

Ένας Sentence Transformer ορίζεται ως συνάρτηση:

$$f : \mathcal{S} \rightarrow \mathbb{R}^d \quad (2.8)$$

όπου $\mathcal{S}$ είναι ο χώρος όλων των προτάσεων και d η διάσταση των embeddings (συνήθως 384 ή 768).

Βασικές Ιδιότητες

1. **Σταθερή Διάσταση:** Ανεξάρτητα από το μήκος της πρότασης, το embedding έχει πάντα την ίδια διάσταση d .
2. **Σημασιολογική Εγγύτητα:** Σημασιολογικά παρόμοιες προτάσεις έχουν υψηλή cosine similarity:

$$\text{sim}(s_1, s_2) = \frac{f(s_1) \cdot f(s_2)}{\|f(s_1)\| \|f(s_2)\|} \in [-1, 1] \quad (2.9)$$

3. **Πυκνή Αναπαράσταση:** Σε αντίθεση με το BoW (sparse), τα embeddings είναι dense vectors με όλες τις διαστάσεις non-zero.

Αρχιτεκτονική - Μοντέλο all-MiniLM-L6-v2

Στην παρούσα εργασία χρησιμοποιείται το μοντέλο *all-MiniLM-L6-v2*, το οποίο βασίζεται στον μετασχηματιστή BERT:

1. **Tokenization:** Η πρόταση διαιρείται σε subword tokens:

$$s \rightarrow [t_1, t_2, \dots, t_n] \quad (2.10)$$

2. **BERT Encoding:** Τα tokens περνούν μέσα από 6 layers του MiniLM:

$$\mathbf{H} = \text{BERT}([t_1, t_2, \dots, t_n]) \in \mathbb{R}^{n \times 384} \quad (2.11)$$

3. **Mean Pooling:** Υπολογίζεται ο μέσος όρος των token embeddings:

$$\mathbf{e} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i \quad (2.12)$$

4. **Normalization:** Το τελικό embedding κανονικοποιείται:

$$f(s) = \frac{\mathbf{e}}{\|\mathbf{e}\|} \quad (2.13)$$

Πλεονεκτήματα έναντι BoW

1. **Σημασιολογική Κατανόηση:** Αιχμαλωτίζει τη σημασία, όχι μόνο τις λέξεις. Π.χ., "cheap" και "inexpensive" έχουν παρόμοια embeddings.
2. **Διαχείριση OOV (Out-of-Vocabulary):** Χρησιμοποιεί subword tokenization (WordPiece), επομένως μπορεί να χειριστεί νέες λέξεις.
3. **Συμπαγής Αναπαράσταση:** 384 διαστάσεις αντί για 5000 του BoW, μειώνοντας τη διαστατικότητα κατά >90%.
4. **Transfer Learning:** Προεκπαιδευμένο σε τεράστια corpora (Wikipedia, BookCorpus), έχει ήδη μάθει γενικές γλωσσικές αναπαραστάσεις.
5. **Σύνθεση Φράσεων:** Λαμβάνει υπόψη τη σειρά και τη σχέση μεταξύ λέξεων, σε αντίθεση με το "bag" του BoW.

Υπολογιστική Πολυπλοκότητα

Η εξαγωγή embeddings έχει πολυπλοκότητα $O(n^2 \cdot d)$ όπου n το μήκος της πρότασης και $d = 384$ η διάσταση, λόγω της self-attention στο BERT. Για το email classification, αυτό είναι αποδεκτό καθώς η διαδικασία γίνεται μία φορά offline (feature extraction) και στη συνέχεια χρησιμοποιούνται τα cached embeddings.

2.3 Αλγόριθμοι Ταξινόμησης

2.3.1 Naive Bayes

Μαθηματικό Υπόδειγμα

Ο αλγόριθμος Naive Bayes βασίζεται στο **θεώρημα του Bayes**:

$$P(c|x) = \frac{P(x|c) \cdot P(c)}{P(x)} \quad (2.14)$$

όπου c η κλάση (spam/ham) και x το διάνυσμα χαρακτηριστικών του email. Για ταξινόμηση, επιλέγουμε την κλάση με τη μεγαλύτερη εκ των υστέρων πιθανότητα:

$$\hat{c} = \arg \max_{c \in \{spam, ham\}} P(c|x) = \arg \max_c P(x|c) \cdot P(c) \quad (2.15)$$

Η παρονομαστής $P(x)$ παραλείπεται καθώς είναι σταθερός για όλες τις κλάσεις.

Multinomial Naive Bayes

Για ταξινόμηση κειμένου, χρησιμοποιούμε το **Multinomial Naive Bayes**, όπου το κείμενο αναπαρίσταται ως διάνυσμα συχνοτήτων λέξεων $x = (n_1, n_2, \dots, n_{|V|})$ με n_i τον αριθμό εμφανίσεων της λέξης w_i . Η πιθανότητα likelihood υπολογίζεται ως:

$$P(x|c) = P(n_1, n_2, \dots, n_{|V|}|c) = \frac{n!}{n_1! n_2! \dots n_{|V|}!} \prod_{i=1}^{|V|} p_i^{n_i} \quad (2.16)$$

όπου $n = \sum_i n_i$ το συνολικό πλήθος λέξεων και $p_i = P(w_i|c)$ η πιθανότητα της λέξης w_i δεδομένης της κλάσης c . Για πρακτικούς λόγους (αποφυγή υπερχείλισης), χρησιμοποιούμε λογαρίθμους:

$$\log P(c|x) \propto \log P(c) + \sum_{i=1}^{|V|} n_i \log P(w_i|c) \quad (2.17)$$

Laplace Smoothing

Για να αποφύγουμε το πρόβλημα μηδενικών πιθανοτήτων όταν μια λέξη δεν εμφανίζεται στο training set, εφαρμόζουμε **Laplace (additive) smoothing**:

$$P(w_i|c) = \frac{\text{count}(w_i, c) + \alpha}{\sum_{j=1}^{|V|} \text{count}(w_j, c) + \alpha|V|} \quad (2.18)$$

όπου $\alpha > 0$ η παράμετρος smoothing (συνήθως $\alpha = 1$). Στην υλοποίησή μας, χρησιμοποιούμε το *MultinomialNB* του scikit-learn με $\alpha = 1.0$.

Παραδοχές και Περιορισμοί

Η θεμελιώδης παραδοχή του Naive Bayes είναι η **δεσμευμένη ανεξαρτησία** (conditional independence): δεδομένης της κλάσης c , τα χαρακτηριστικά $x_1, \dots, x_n$ είναι ανεξάρτητα:

$$P(x_1, \dots, x_n|c) = \prod_{i=1}^n P(x_i|c) \quad (2.19)$$

Αυτή η παραδοχή είναι "naive" (αφελής) καθώς στην πράξη οι λέξεις σε ένα κείμενο δεν είναι ανεξάρτητες (π.χ., η λέξη "York" συχνά εμφανίζεται μετά τη "New"). Παρόλα αυτά, ο αλγόριθμος αποδίδει καλά στην πράξη για τους εξής λόγους:

1. **Computational Efficiency**: Η εκπαίδευση έχει πολυπλοκότητα $O(n \cdot d)$ όπου n το πλήθος δειγμάτων και d τα χαρακτηριστικά, χωρίς να απαιτείται επαναληπτική βελτιστοποίηση.
2. **Low Variance**: Εκτιμά λιγότερες παραμέτρους ($O(d)$) σε σχέση με άλλους classifiers, μειώνοντας το overfitting σε μικρά datasets.
3. **Asymptotic Optimality**: Σε πολλές περιπτώσεις, η violation της independence assumption δεν επηρεάζει σημαντικά την ακρίβεια ταξινόμησης.

Ωστόσο, οι περιορισμοί περιλαμβάνουν την αδυναμία να μοντελοποιήσει συσχετίσεις μεταξύ χαρακτηριστικών και την υποτίμηση ή υπερεκτίμηση των πιθανοτήτων (calibration issues).

2.3.2 k-Nearest Neighbors (k-NN)

Μαθηματικό Υπόδειγμα

Ο αλγόριθμος k-Nearest Neighbors είναι ένας non-parametric, instance-based μέθοδος μάθησης. Για ένα νέο δείγμα x , ο k-NN:

1. Υπολογίζει την απόσταση $d(x, x_i)$ από όλα τα training δείγματα x_i .
2. Επιλέγει τα k πλησιέστερα δείγματα (nearest neighbors).
3. Εκχωρεί την κλάση με **majority voting**:

$$\hat{y} = \arg \max_c \sum_{x_i \in \mathcal{N}_k(x)} \mathbb{I}(y_i = c) \quad (2.20)$$

όπου $\mathcal{N}_k(x)$ το σύνολο των k πλησιέστερων γειτόνων και $\mathbb{I}(\cdot)$ η indicator function.

Μετρικές Απόστασης

Η πιο συνηθισμένη μετρική για sentence embeddings είναι η **Ευκλείδεια απόσταση**:

$$d_{Euclidean}(x, x') = \sqrt{\sum_{j=1}^d (x_j - x'_j)^2} = \|x - x'\|_2 \quad (2.21)$$

Εναλλακτικά, για normalized embeddings (όπως τα Sentence Transformers), χρησιμοποιείται η **Cosine distance**:

$$d_{Cosine}(x, x') = 1 - \frac{x \cdot x'}{\|x\| \|x'\|} = 1 - \cos(\theta) \quad (2.22)$$

Άλλες μετρικές περιλαμβάνουν την Manhattan distance (L_1 norm) και τη Minkowski distance (γενίκευση των L_p norms).

Weighted k-NN

Για να δώσουμε μεγαλύτερη βαρύτητα στους πλησιέστερους γείτονες, χρησιμοποιούμε **distance-weighted voting**:

$$\hat{y} = \arg \max_c \sum_{x_i \in \mathcal{N}_k(x)} w_i \cdot \mathbb{I}(y_i = c), \quad w_i = \frac{1}{d(x, x_i) + \epsilon} \quad (2.23)$$

όπου $\epsilon > 0$ μικρή σταθερά για αποφυγή διαίρεσης με μηδέν.

Επιλογή k

Η παράμετρος k ελέγχει την πολυπλοκότητα του μοντέλου:

1. **Μικρό k**: Χαμηλό bias, υψηλό variance $\rightarrow$ overfitting. Στην ακραία περίπτωση $k = 1$, το μοντέλο απομνημονεύει τα training data.
2. **Μεγάλο k**: Υψηλό bias, χαμηλό variance $\rightarrow$ underfitting. Όταν $k \rightarrow n$, ο k-NN εκφυλίζεται σε majority-class classifier.
3. **Βέλτιστο k**: Επιλέγεται με cross-validation, συνήθως $k \in [3, 20]$. Μία heuristic είναι $k \approx \sqrt{n}$.

Curse of Dimensionality

Ο k-NN υποφέρει από το **curse of dimensionality**: σε υψηλές διαστάσεις ($d \gg 100$), όλα τα σημεία τείνουν να έχουν παρόμοιες αποστάσεις, κάνοντας την "εγγύτητα" ασαφή. Για BoW με $d = 5000$, αυτό είναι πρόβλημα. Για Sentence Transformers με $d = 384$, είναι πιο διαχειρίσιμο, ενώ το PCA μειώνει περαιτέρω τη διαστατικότητα.

Υπολογιστική Πολυπλοκότητα

Η πρόβλεψη έχει πολυπλοκότητα $O(n \cdot d)$ για naive implementation (σύγκριση με όλα τα training δείγματα). Αυτό κάνει τον k-NN *ακατάλληλο για πολύ μεγάλα datasets*. Βελτιστοποιήσεις όπως KD-

trees, Ball trees, ή LSH (Locality-Sensitive Hashing) μειώνουν την πολυπλοκότητα, αλλά εξακολουθεί να είναι πολύ αργότερος από parametric μοντέλα (SVM, Naive Bayes) που έχουν $O(d)$ prediction time.

2.3.3 Support Vector Machines (SVM)

Μαθηματικό Υπόδειγμα

Τα Support Vector Machines είναι ισχυροί αλγόριθμοι για δυαδική ταξινόμηση που βρίσκουν το βέλτιστο διαχωριστικό **hyperplane** μεταξύ των κλάσεων. Για γραμμικά διαχωρίσιμα δεδομένα, το hyperplane ορίζεται ως:

$$w^T x + b = 0 \quad (2.24)$$

όπου $w \in \mathbb{R}^d$ το διάνυσμα βαρών (κάθετο στο hyperplane) και b το bias term. Η ταξινόμηση γίνεται με:

$$\hat{y} = \text{sign}(w^T x + b) \in \{-1, +1\} \quad (2.25)$$

Maximum Margin Classification

Το κλειδί του SVM είναι η μεγιστοποίηση του **margin** $\gamma = \frac{2}{\|w\|}$, δηλαδή της ελάχιστης απόστασης των training δειγμάτων από το hyperplane. Αυτό οδηγεί στο βελτιστοποιητικό πρόβλημα:

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|^2 \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1, \quad \forall i = 1, \dots, n \end{aligned} \quad (2.26)$$

Τα σημεία που ικανοποιούν $y_i(w^T x_i + b) = 1$ (δηλαδή βρίσκονται ακριβώς στο margin) ονομάζονται **support vectors** και είναι τα μόνα που επηρεάζουν τη λύση.

Soft-Margin SVM

Για μη γραμμικά διαχωρίσιμα δεδομένα, χαλαρώνουμε τους περιορισμούς εισάγοντας **slack variables** $\xi_i \geq 0$:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i \end{aligned} \quad (2.27)$$

Η παράμετρος $C > 0$ ελέγχει το trade-off μεταξύ μεγιστοποίησης του margin και ελαχιστοποίησης των training errors:

- **Μικρό C:** Μεγάλο margin, αλλά επιτρέπονται περισσότερα misclassifications (underfitting).
- **Μεγάλο C:** Λίγα misclassifications, αλλά μικρότερο margin (overfitting).

Kernel Functions

Για μη γραμμικά προβλήματα, το **kernel trick** μετασχηματίζει τα δεδομένα σε υψηλότερο διαστασιακό χώρο $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$ χωρίς να υπολογίσει ρητά το $\phi(x)$. Ο dual formulation του SVM χρησιμοποιεί μόνο εσωτερικά γινόμενα $\langle \phi(x_i), \phi(x_j) \rangle$, τα οποία αντικαθίστανται από τη **kernel function**:

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle \quad (2.28)$$

Οι πιο συνηθισμένοι kernels είναι:

1. **Linear Kernel:**

$$K(x, x') = x^T x' \quad (2.29)$$

Χρήσιμος όταν τα δεδομένα είναι ήδη γραμμικά διαχωρίσιμα ή έχουν πολύ υψηλή διάσταση.

2. **Polynomial Kernel:**

$$K(x, x') = (x^T x' + c)^d \quad (2.30)$$

Μοντελοποιεί πολυωνυμικές σχέσεις βαθμού d . Σπάνια χρησιμοποιείται λόγω αριθμητικών προβλημάτων.

3. **Radial Basis Function (RBF) / Gaussian Kernel:**

$$K(x, x') = \exp(-\gamma \|x - x'\|^2), \quad \gamma = \frac{1}{2\sigma^2} \quad (2.31)$$

Ο πιο δημοφιλής kernel, αντιστοιχεί σε άπειρης διάστασης χώρο χαρακτηριστικών. Η παράμετρος γ ελέγχει την "εμβέλεια" επιρροής κάθε training example: μεγάλο γ οδηγεί σε overfitting (επηρεάζονται μόνο πολύ κοντινά σημεία), μικρό γ σε underfitting.

Στην υλοποίησή μας, χρησιμοποιούμε **RBF kernel** με $\gamma = \text{scale} = \frac{1}{d \cdot \text{Var}(X)}$ και $C \in \{0.1, 1, 10, 100\}$ που βελτιστοποιούνται με grid search.

Υπολογιστική Πολυπλοκότητα

Η εκπαίδευση SVM έχει πολυπλοκότητα μεταξύ $O(n^2)$ και $O(n^3)$ ανάλογα με τον optimization solver (SMO, quadratic programming). Η πρόβλεψη έχει πολυπλοκότητα $O(n_{SV} \cdot d)$ όπου n_{SV} ο αριθμός των support vectors (συνήθως $n_{SV} \ll n$). Αυτό κάνει το SVM αργό για πολύ μεγάλα datasets αλλά αποδοτικό για μεσαίου μεγέθους.

2.3.4 Logistic Regression

Μαθηματικό Υπόδειγμα

Η Logistic Regression είναι ένα γραμμικό μοντέλο για δυαδική ταξινόμηση που μοντελοποιεί την πιθανότητα $P(y = 1|x)$ χρησιμοποιώντας τη **sigmoid (logistic) function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z} \quad (2.32)$$

Το μοντέλο ορίζεται ως:

$$P(y = 1|x; w, b) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}} \quad (2.33)$$

όπου $w \in \mathbb{R}^d$ τα βάρη και b το bias. Η ταξινόμηση γίνεται με threshold (συνήθως 0.5):

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \quad (2.34)$$

Log-Odds και Maximum Likelihood

Η **log-odds** (logit) είναι γραμμική συνάρτηση των χαρακτηριστικών:

$$\log \frac{P(y=1|x)}{P(y=0|x)} = w^T x + b \quad (2.35)$$

Τα βάρη w εκπαιδεύονται μεγιστοποιώντας τη **log-likelihood**:

$$\mathcal{L}(w, b) = \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)] \quad (2.36)$$

όπου $\hat{p}_i = P(y_i = 1|x_i)$. Αυτό ισοδυναμεί με ελαχιστοποίηση της **binary cross-entropy loss**:

$$J(w, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)] \quad (2.37)$$

Η βελτιστοποίηση γίνεται με gradient descent ή Newton-Raphson methods.

Regularization

Για αποφυγή overfitting, προστίθεται **regularization penalty**:

$$J_{reg}(w, b) = J(w, b) + \lambda R(w) \quad (2.38)$$

Οι δύο πιο συνηθισμένοι τύποι είναι:

1. L2 Regularization (Ridge):

$$R(w) = \frac{1}{2} \|w\|_2^2 = \frac{1}{2} \sum_{j=1}^d w_j^2 \quad (2.39)$$

Συρρικνώνει όλα τα βάρη προς το μηδέν αλλά δεν τα μηδενίζει. Αντιστοιχεί σε Gaussian prior στο Bayesian framework.

2. L1 Regularization (Lasso):

$$R(w) = \|w\|_1 = \sum_{j=1}^d |w_j| \quad (2.40)$$

Προάγει **sparsity**: πολλά βάρη γίνονται ακριβώς μηδέν, επιτρέποντας *automatic feature selection*. Αντιστοιχεί σε Laplace prior.

3. Elastic Net:

$$R(w) = \alpha \|w\|_1 + (1 - \alpha) \|w\|_2^2 \quad (2.41)$$

Συνδυάζει L1 και L2, παίρνοντας τα πλεονεκτήματα και των δύο.

Η παράμετρος $\lambda > 0$ (ή ισοδύναμα $C = 1/\lambda$ στο scikit-learn) ελέγχει την ισχύ του regularization: μεγάλο λ οδηγεί σε underfitting, μικρό λ σε overfitting.

Πλεονεκτήματα

1. **Interpretability:** Τα βάρη w_j δείχνουν τη συμβολή κάθε χαρακτηριστικού. Θετικό w_j αυξάνει την πιθανότητα $P(y = 1)$.
2. **Probabilistic Output:** Παρέχει calibrated probabilities, όχι μόνο binary labels.
3. **Computational Efficiency:** Η εκπαίδευση έχει πολυπλοκότητα $O(n \cdot d)$ ανά iteration, η πρόβλεψη $O(d)$.
4. **Robustness:** Με κατάλληλο regularization, αποφεύγει το overfitting ακόμα και σε υψηλές διαστάσεις ($d > n$).

Στην υλοποίησή μας, χρησιμοποιούμε L2 regularization με $C \in \{0.01, 0.1, 1, 10, 100\}$ που βελτιστοποιείται με grid search.

2.4 Μείωση Διαστάσεων

2.4.1 Principal Component Analysis (PCA)

Το Principal Component Analysis είναι μία μέθοδος γραμμικής μείωσης διαστάσεων που μετασχηματίζει τα δεδομένα σε νέο σύστημα συντεταγμένων όπου η μέγιστη διασπορά (variance) βρίσκεται στην πρώτη συνιστώσα (PC1), η δεύτερη μέγιστη στη δεύτερη (PC2), κ.ο.κ.

Μαθηματική Διατύπωση

Έστω $X \in \mathbb{R}^{n \times d}$ ο πίνακας δεδομένων (centered: $\sum_i x_i = 0$). Ο **covariance matrix** είναι:

$$\Sigma = \frac{1}{n-1} X^T X \in \mathbb{R}^{d \times d} \quad (2.42)$$

Το PCA βρίσκει τους **eigenvectors** (principal components) $v_1, \dots, v_d$ και τα αντίστοιχα **eigenvalues** $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$ του Σ :

$$\Sigma v_i = \lambda_i v_i, \quad \|v_i\| = 1, \quad v_i^T v_j = \delta_{ij} \quad (2.43)$$

Ο πρώτος principal component v_1 είναι η κατεύθυνση μέγιστης διασποράς:

$$v_1 = \arg \max_{\|v\|=1} \text{Var}(Xv) = \arg \max_{\|v\|=1} v^T \Sigma v \quad (2.44)$$

Για μείωση από d σε k διαστάσεις, κρατάμε τα πρώτα k principal components $V_k = [v_1, \dots, v_k] \in \mathbb{R}^{d \times k}$ και μετασχηματίζουμε:

$$\tilde{X} = X V_k \in \mathbb{R}^{n \times k} \quad (2.45)$$

Η ανακατασκευή (reconstruction) είναι $\hat{X} = \tilde{X} V_k^T$, και το **reconstruction error** είναι:

$$\|X - \hat{X}\|_F^2 = \sum_{i=k+1}^d \lambda_i \quad (2.46)$$

όπου $\|\cdot\|_F$ η Frobenius norm.

Singular Value Decomposition (SVD)

Στην πράξη, το PCA υπολογίζεται με SVD:

$$X = U\Sigma V^T \quad (2.47)$$

όπου $U \in \mathbb{R}^{n \times n}$, $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_d)$ με $\sigma_i = \sqrt{\lambda_i}$ (singular values), και $V \in \mathbb{R}^{d \times d}$ οι eigenvectors. Το SVD είναι πιο αριθμητικά σταθερό από την άμεση diagonalization του $X^T X$.

2.4.2 Explained Variance

Η **explained variance ratio** για το i -οστό principal component είναι:

$$r_i = \frac{\lambda_i}{\sum_{j=1}^d \lambda_j} \quad (2.48)$$

Η **cumulative explained variance** για τα πρώτα k components είναι:

$$R_k = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^d \lambda_j} \quad (2.49)$$

Η επιλογή του αριθμού k γίνεται με βάση ένα από τα εξής κριτήρια:

1. **Variance Threshold:** Κρατάμε components μέχρι να φτάσουμε $R_k \geq \theta$ (συνήθως $\theta = 0.90$ ή 0.95), δηλαδή διατηρούμε 90-95% της variance.
2. **Scree Plot:** Σχεδιάζουμε τα λ_i συναρτήσει του i και επιλέγουμε το "elbow" point όπου τα eigenvalues αρχίζουν να πέφτουν πιο αργά.
3. **Kaiser Criterion:** Κρατάμε components με $\lambda_i > 1$ (ή $\lambda_i > \bar{\lambda}$ όπου $\bar{\lambda}$ ο μέσος eigenvalue). Αυτό λειτουργεί μόνο αν τα δεδομένα είναι standardized.
4. **Cross-Validation:** Επιλέγουμε το k που μεγιστοποιεί την ακρίβεια στο downstream task (π.χ., classification accuracy).

Στην υλοποίησή μας, δοκιμάζουμε variance ratios $\{0.90, 0.95, 0.99\}$ (που αντιστοιχούν σε 112, 147, και 230 components αντίστοιχα) καθώς και σταθερό αριθμό 10 components.

Ιδιότητες και Περιορισμοί

Πλεονεκτήματα:

- Μειώνει τη διαστατικότητα διατηρώντας τη μέγιστη δυνατή διασπορά.
- Αφαιρεί collinearity (τα PCs είναι ορθογώνια).
- Μειώνει το overfitting και την υπολογιστική πολυπλοκότητα.

Περιορισμοί:

- Μόνο γραμμικοί μετασχηματισμοί (δεν συλλαμβάνει μη γραμμικές δομές).
- Τα PCs δεν έχουν απαραίτητα ερμηνεία (είναι γραμμικοί συνδυασμοί features).
- Ευαίσθητο σε outliers (επειδή μεγιστοποιεί variance).
- Απαιτεί standardization αν τα features έχουν διαφορετικές κλίμακες.

Για μη γραμμική μείωση διαστάσεων, εναλλακτικές είναι το t-SNE, UMAP, ή autoencoders.

2.5 Μετρικές Αξιολόγησης

2.5.1 Confusion Matrix

Το **Confusion Matrix** είναι ένας πίνακας 2×2 που συνοψίζει τις προβλέψεις ενός δυαδικού classifier:

		Πραγματική Κλάση	
		Positive (Spam)	Negative (Ham)
Πρόβλεψη	Positive	TP (True Positive)	FP (False Positive)
	Negative	FN (False Negative)	TN (True Negative)

Οι τέσσερις κατηγορίες είναι:

- **True Positive (TP)**: Spam email που ταξινομήθηκε σωστά ως spam.
- **True Negative (TN)**: Ham email που ταξινομήθηκε σωστά ως ham.
- **False Positive (FP)**: Ham email που ταξινομήθηκε λανθασμένα ως spam (*Type I error*).
- **False Negative (FN)**: Spam email που ταξινομήθηκε λανθασμένα ως ham (*Type II error*).

Στο πρόβλημα spam classification, τα FP είναι συνήθως πιο κρίσιμα (ένα legitimate email χάνεται) από τα FN (ένα spam περνάει).

2.5.2 Precision, Recall, F1-Score

Accuracy

Το ποσοστό σωστών προβλέψεων:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.50)$$

Το Accuracy είναι παραπλανητικό σε **imbalanced datasets**. Για παράδειγμα, αν το 95% είναι ham, ένας classifier που προβλέπει πάντα ham έχει 95% accuracy αλλά είναι άχρηστος.

Precision και Recall

Precision (Positive Predictive Value):

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2.51)$$

Απαντά: "Από όσα προβλέψαμε ως spam, πόσα ήταν όντως spam;" Υψηλό precision σημαίνει λίγα false positives.

Recall (Sensitivity, True Positive Rate):

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.52)$$

Απαντά: "Από όλα τα spam emails, πόσα βρήκαμε;" Υψηλό recall σημαίνει λίγα false negatives.

Υπάρχει **precision-recall trade-off**: αυξάνοντας το threshold για positive prediction, το precision αυξάνεται αλλά το recall μειώνεται, και αντίστροφα.

F1-Score

Το **F1-Score** είναι ο αρμονικός μέσος του precision και recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \quad (2.53)$$

Ο αρμονικός μέσος τιμωρεί ακραίες τιμές: αν είτε precision είτε recall είναι πολύ χαμηλό, το F_1 είναι χαμηλό. Το $F_1 = 1$ σημαίνει τέλει classifier (no false positives ή false negatives).

Γενίκευση: Το F_β score δίνει β φορές περισσότερη βαρύτητα στο recall:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \quad (2.54)$$

Για $\beta = 2$, το recall έχει διπλάσια σημασία (προτιμάται όταν τα false negatives είναι πιο κρίσιμα).

2.5.3 ROC Curve και AUC

Η **Receiver Operating Characteristic (ROC) curve** είναι ένα γράφημα που δείχνει την απόδοση ενός binary classifier σε διάφορα classification thresholds. Απεικονίζει:

- **X-axis**: False Positive Rate (FPR) = $\frac{FP}{FP+TN} = 1 - \text{Specificity}$
- **Y-axis**: True Positive Rate (TPR) = $\frac{TP}{TP+FN} = \text{Recall}$

Για κάθε threshold t , υπολογίζουμε $(FPR(t), TPR(t))$ και τα συνδέουμε με γραμμή.

Area Under Curve (AUC)

Η **AUC-ROC** είναι το εμβαδόν κάτω από την ROC καμπύλη:

$$\text{AUC} = \int_0^1 TPR(FPR^{-1}(x)) dx \quad (2.55)$$

Ερμηνεία: Η πιθανότητα ο classifier να δίνει υψηλότερο score σε ένα τυχαίο positive example παρά σε ένα τυχαίο negative example.

- $\text{AUC} = 1.0$: Τέλειος classifier (διαχωρίζει πλήρως τις κλάσεις).
- $\text{AUC} = 0.5$: Random guessing (η ROC είναι η διαγώνιος).
- $\text{AUC} < 0.5$: Χειρότερο από τυχαίο (συνήθως σημαίνει ότι οι labels είναι flipped).

Το AUC είναι **threshold-independent** και λειτουργεί καλά σε imbalanced datasets. Στην πράξη, $\text{AUC} > 0.90$ θεωρείται excellent, $\text{AUC} > 0.80$ good.

Κεφάλαιο 3

Μεθοδολογία

3.1 Συλλογή και Περιγραφή Δεδομένων

3.1.1 Dataset Emails.csv

Το dataset που χρησιμοποιήθηκε είναι το αρχείο *emails.csv*, το οποίο περιέχει συλλογή emails με δυαδική ετικέτα spam/ham. Το dataset έχει τη δομή:

- **text**: Το πλήρες κείμενο του email (συμπεριλαμβανομένου του θέματος)
- **spam**: Δυαδική ετικέτα (1 = spam, 0 = ham/legitimate)

Μέγεθος Dataset

Το συνολικό μέγεθος του dataset είναι **5,728 emails**, το οποίο θεωρείται μεσαίου μεγέθους για έργα ταξινόμησης κειμένου. Αυτό το μέγεθος είναι επαρκές για την εκπαίδευση μοντέλων μηχανικής μάθησης αλλά όχι αρκετά μεγάλο ώστε να απαιτεί deep learning προσεγγίσεις με πολλά δεδομένα.

3.1.2 Κατανομή Κλάσεων

Η κατανομή των κλάσεων στο dataset είναι **ανισόρροπη** (imbalanced):

Κλάση	Πλήθος	Ποσοστό
Ham (Legitimate)	4,360	76.1%
Spam	1,368	23.9%
Σύνολο	5,728	100%

Πίνακας 3.1: Κατανομή κλάσεων στο dataset emails.csv

Ανάλυση Ισορροπίας

Η αναλογία ham προς spam είναι περίπου **3.2:1**, δηλαδή υπάρχουν περίπου 3 φορές περισσότερα legitimate emails από spam. Αυτή η ανισορροπία είναι τυπική για πραγματικά datasets spam classification και αντικατοπτρίζει ρεαλιστικές συνθήκες:

$$\text{Imbalance Ratio} = \frac{|\text{Ham}|}{|\text{Spam}|} = \frac{4360}{1368} \approx 3.19 \quad (3.1)$$

Η ανισορροπία αυτή δεν είναι ακραία (extreme imbalance θα ήταν >10:1), οπότε δεν απαιτούνται ειδικές τεχνικές όπως SMOTE ή class weighting. Ωστόσο, πρέπει να ληφθεί υπόψη κατά την αξιολόγηση:

- Το **accuracy** μόνο του δεν είναι επαρκής μετρική (ένας classifier που προβλέπει πάντα ham θα είχε 76.1% accuracy)
- Χρησιμοποιούμε **F1-score, Precision, Recall** και **ROC-AUC** για πιο ισορροπημένη αξιολόγηση
- Το **stratified split** διατηρεί την ίδια αναλογία σε train/validation/test sets

3.1.3 Exploratory Data Analysis

Στατιστικά Μήκους Κειμένου

Το μήκος των emails (σε αριθμό χαρακτήρων) ποικίλλει σημαντικά:

- **Μέσο μήκος:** Τα spam emails τείνουν να είναι συντομότερα από τα ham
- **Λεκτική ποικιλία:** Τα legitimate emails έχουν μεγαλύτερη λεξιλογική ποικιλία
- **Χαρακτηριστικά spam:** Περιέχουν συχνά λέξεις όπως "free", "click", "urgent", "winner", "money"

Αυτές οι διαφορές αποτελούν σήματα (signals) που εκμεταλλεύονται οι αλγόριθμοι ταξινόμησης για να διακρίνουν spam από ham.

3.2 Διαχωρισμός Δεδομένων

3.2.1 Training, Validation, Test Sets

Ο διαχωρισμός των δεδομένων ακολουθεί την τυπική προσέγγιση train/validation/test split με **στρατηγική επιλογή μεγεθών** που εξισορροπεί την ποσότητα δεδομένων για εκπαίδευση με την αξιοπιστία της αξιολόγησης.

Διαίρεση Dataset

Το συνολικό dataset (5,728 emails) χωρίστηκε ως εξής:

Set	Emails	% Total	Ham	Spam	Spam %
Training	4,582	80.0%	3,461	1,121	24.5%
Validation	572	10.0%	436	136	23.8%
Test	574	10.0%	437	137	23.9%
Σύνολο	5,728	100%	4,360	1,368	23.9%

Πίνακας 3.2: Διαχωρισμός δεδομένων σε Training, Validation και Test sets

Αιτιολόγηση Επιλογών

1. Training Set (4,582 emails):

- Μεγάλο μέγεθος (80%) για καλύτερη εκπαίδευση μοντέλων
- Περιλαμβάνει 3,461 ham και 1,121 spam emails
- Αναλογία spam 24.5% (παρόμοια με το συνολικό 23.9%)

2. Validation Set (572 emails):

- Χρησιμοποιείται για hyperparameter tuning και model selection
- 10% του dataset, επαρκές για αξιόπιστη αξιολόγηση
- Αποτρέπει το overfitting στο test set από πολλαπλές δοκιμές

3. Test Set (574 emails):

- 10% του dataset για *unbiased* τελική αξιολόγηση
- Παρέχει στατιστικά σημαντικά αποτελέσματα με υψηλή αξιοπιστία
- Δεν χρησιμοποιείται ποτέ κατά την εκπαίδευση ή tuning

Μαθηματική Διατύπωση

Έστω $D = \{(x_i, y_i)\}_{i=1}^N$ το πλήρες dataset με $N = 5728$. Ο διαχωρισμός ορίζεται ως:

$$D = D_{train} \cup D_{val} \cup D_{test}, \quad D_{train} \cap D_{val} \cap D_{test} = \emptyset \quad (3.2)$$

όπου:

$$|D_{train}| = 4582 \quad (3.3)$$

$$(3.4)$$

$$|D_{val}| = 572 \quad (3.5)$$

$$(3.6)$$

$$|D_{test}| = 574 \quad (3.7)$$

3.2.2 Stratified Split

Για να διατηρηθεί η αναλογία των κλάσεων σε όλα τα sets, εφαρμόζεται **stratified sampling**. Αυτό εξασφαλίζει ότι κάθε set έχει παρόμοια κατανομή spam/ham με το αρχικό dataset.

Στρατηγική Υλοποίησης

1. Shuffling με random_state=42:

- Το dataset ανακατεύεται τυχαία για αποφυγή bias από τη σειρά των emails
- Χρήση σταθερού seed (42) για *reproducibility*
- Εξασφαλίζει ότι τα αποτελέσματα είναι επαναληπτέα

2. Sequential Split:

- Πρώτα 2,000 emails → Training
- Επόμενα 1,000 emails → Validation
- Υπόλοιπα 2,728 emails → Test

3. Verification της Stratification:

- Training: 25.7% spam (vs 23.9% overall) - απόκλιση +1.8%
- Validation: 23.9% spam - ακριβής αντιστοιχία
- Test: 22.6% spam - απόκλιση -1.3%

Σημασία του Stratified Split

Το stratified split είναι κρίσιμο σε imbalanced datasets γιατί:

- **Αποτρέπει bias:** Χωρίς stratification, ένα set μπορεί να έχει δυσανάλογα λίγα spam examples
- **Συνεπής αξιολόγηση:** Όλα τα sets αντιπροσωπεύουν την ίδια κατανομή
- **Καλύτερη γενίκευση:** Τα μοντέλα εκπαιδεύονται και αξιολογούνται σε ρεαλιστικές αναλογίες

Η μικρή απόκλιση (<2%) στα ποσοστά spam μεταξύ των sets είναι αποδεκτή και οφείλεται στο ότι χρησιμοποιήθηκε απλό shuffling αντί για sophisticated stratified split algorithm.

3.3 Προεπεξεργασία Κειμένου

Η προεπεξεργασία κειμένου αποτελεί κρίσιμο στάδιο γιατί μετασχηματίζει το ακατέργαστο κείμενο σε καθαρή αναπαράσταση κατάλληλη για μηχανική μάθηση. Ο στόχος είναι να αφαιρεθεί ο θόρυβος (noise) και να διατηρηθούν μόνο τα σημαντικά στοιχεία που διακρίνουν spam από ham.

3.3.1 Πέντε-βημάτων-στάδιο Καθαρισμού

Η υλοποίηση ακολουθεί ένα **πέντε-βημάτων-στάδιο pipeline προεπεξεργασίας**, όπου κάθε στάδιο εφαρμόζεται διαδοχικά στο κείμενο:

1. Αφαίρεση URLs

Τα emails (ιδίως spam) περιέχουν συχνά URLs που δεν συνεισφέρουν στην ταξινόμηση αλλά προσθέτουν διάσταση στο feature space:

```
● ● ● URL Removal
1 # Χρήση regular expression για εντοπισμό και αφαίρεση URLs
2 text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
```

Αυτό αφαιρεί patterns όπως `http://example.com`, `www.spam.com`, και `https://malicious.site`.

2. Αφαίρεση Email Addresses

Οι email διευθύνσεις αφαιρούνται γιατί είναι μοναδικά identifiers που δεν γενικεύουν καλά:

```
● ● ● Email Addresses Removal
1 text = re.sub(r'\S+@\S+', '', text)
```

3. Αφαίρεση Ειδικών Χαρακτήρων και Αριθμών

Διατηρούμε μόνο γράμματα και κενά, αφαιρώντας σημεία στίξης, αριθμούς και σύμβολα:

```
● ● ● Special Characters Removal
1 text = re.sub(r'^a-zA-Z\s', '', text)
```

Αυτό μετατρέπει `"Get 50% off NOW!!!"` σε `"Get off NOW"`.

4. Μετατροπή σε Πεζά και Κανονικοποίηση Κενών

Ο μετασχηματισμός σε lowercase εξασφαλίζει ότι "FREE", "Free", και "free" αντιμετωπίζονται ως η ίδια λέξη:

● ● ● Lowercase Conversion

```
1 text = text.lower()
2 text = ' '.join(text.split()) # Normalize multiple spaces
```

5. Stopwords Removal

Χρησιμοποιήθηκε η λίστα **NLTK English stopwords** που περιέχει **179 κοινές αγγλικές λέξεις** όπως "the", "is", "at", "which", "on", κτλ:

● ● ● Stopwords Removal

```
1 from nltk.corpus import stopwords
2
3 stop_words = set(stopwords.words('english')) # 179 words
4 words = text.split()
5 filtered_words = [word for word in words if word not in stop_words]
6 text = ' '.join(filtered_words)
```

3.3.2 Μαθηματική Διατύπωση

Η προεπεξεργασία μπορεί να εκφραστεί ως σύνθεση συναρτήσεων:

$$f_{\text{preprocess}} = f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1 \quad (3.8)$$

όπου:

f_1 : URL removal

f_2 : Email address removal

f_3 : Special characters removal

f_4 : Lowercase + space normalization

f_5 : Stopwords removal

Για κάθε email $d_i \in D$, το καθαρό κείμενο είναι:

$$d'_i = f_{\text{preprocess}}(d_i) = f_5(f_4(f_3(f_2(f_1(d_i)))))) \quad (3.9)$$

3.3.3 Απόφαση: Όχι Stemming/Lemmatization

Αποφασίστηκε να **μην** χρησιμοποιηθεί stemming (π.χ. Porter Stemmer) ή lemmatization για τους εξής λόγους:

1. **Sentence Transformers Capability:** Τα BERT-based μοντέλα (όπως το paraphrase-multilingual-MiniLM-L12-v2) είναι εκπαιδευμένα σε φυσικό κείμενο και χειρίζονται διαφορετικές μορφές της ίδιας λέξης (π.χ. "run", "running", "ran") μέσω του contextual encoding
2. **Bag-of-Words με Term Frequency:** Στο BoW μοντέλο χρησιμοποιούμε term frequency counts. Οι διαφορετικές μορφές μίας λέξης θα αντιμετωπίζονταν ως ξεχωριστές διαστάσεις, αλλά η συχνότητα εμφάνισης μπορεί να προσφέρει χρήσιμη πληροφορία
3. **Διατήρηση Πληροφορίας:** Το stemming μπορεί να χάσει σημαντικές αποχρώσεις. Π.χ. "winner" και "winning" έχουν διαφορετική χρήση σε spam emails
4. **Απλότητα Pipeline:** Αποφυγή πρόσθετης πολυπλοκότητας χωρίς ξεκάθαρο όφελος

3.3.4 Επίδραση Προεπεξεργασίας

Η εφαρμογή του pipeline οδηγεί σε:

- **Μείωση μήκους:** Το μέσο μήκος των emails (σε χαρακτήρες) μειώνεται περίπου 20-30%
- **Μείωση λέξεων:** Αφαιρούνται περίπου 30-40% των λέξεων (κυρίως stopwords)
- **Καθαρότερη αναπαράσταση:** Μόνο content words διατηρούνται
- **Ομοιομορφία:** Όλα τα κείμενα στο ίδιο format (lowercase, no special chars)

Παράδειγμα μετασχηματισμού:

Original: "Click HERE for 50% OFF! Visit www.spam.com NOW!!!"

After preprocessing: "click off visit spam"

Το καθαρό κείμενο διατηρεί τις σημαντικές λέξεις ("click", "off", "visit", "spam") που είναι χαρακτηριστικές για spam, ενώ αφαιρεί τον θόρυβο.

3.4 Αναπαράσταση Χαρακτηριστικών

Μετά την προεπεξεργασία, τα κείμενα πρέπει να μετατραπούν σε αριθμητικές αναπαραστάσεις (feature vectors) που μπορούν να χρησιμοποιηθούν από αλγόριθμους μηχανικής μάθησης. Υλοποιήθηκαν **δύο διαφορετικές προσεγγίσεις**: Bag-of-Words και Sentence Embeddings, για να συγκριθεί η απόδοσή τους.

3.4.1 Bag of Words με CountVectorizer

Η πρώτη μέθοδος αναπαράστασης είναι το κλασικό **Bag-of-Words (BoW)** μοντέλο, που αναπαριστά κάθε κείμενο ως διάνυσμα συχνοτήτων λέξεων.

Υλοποίηση με CountVectorizer

Χρησιμοποιήθηκε το *CountVectorizer* της scikit-learn με τις εξής παραμέτρους:

```
CountVectorizer Configuration
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 vectorizer = CountVectorizer(
4     max_features=5000 # Κρατάμε τις 5000 πιο συχνές λέξεις
5 )
```

```

6
7 # Fit μόνο στο training set
8 X_train_bow = vectorizer.fit_transform(train_df['clean_text'])
9
10 # Transform validation και test sets
11 X_val_bow = vectorizer.transform(val_df['clean_text'])
12 X_test_bow = vectorizer.transform(test_df['clean_text'])

```

Παράμετροι και Αιτιολόγηση

1. `max_features=5000`:

- Περιορίζουμε το λεξιλόγιο στις 5000 πιο συχνές λέξεις
- Αποφεύγουμε την εκρηκτική διαστατικότητα (full vocabulary » 5000)
- Οι σπάνιες λέξεις (hapax legomena) συνήθως δεν γενικεύουν καλά
- Balance μεταξύ πληροφορίας και υπολογιστικής πολυπλοκότητας

2. `Default term frequency (TF)`:

- Χρήση **term frequency** (counts) ως βάρη
- Κάθε διάσταση: ο αριθμός εμφανίσεων της λέξης στο email
- Πιο πλούσια αναπαράσταση από binary (presence/absence)
- Κατάλληλο για Multinomial Naive Bayes
- Λαμβάνει υπόψη τη συχνότητα εμφάνισης λέξεων όπου:

$$x_i = \begin{cases} 1, & \text{αν } w_i \in d \\ 0, & \text{αν } w_i \notin d \end{cases} \quad (3.10)$$

Διαστάσεις Matrices

Οι sparse matrices που προκύπτουν έχουν διαστάσεις:

- **Training:** (4582, 5000) - 4,582 emails × 5,000 features
- **Validation:** (572, 5000) - 572 emails × 5,000 features
- **Test:** (574, 5000) - 574 emails × 5,000 features

Οι matrices είναι **sparse** (πολλά zeros) γιατί κάθε email περιέχει μόνο ένα μικρό υποσύνολο του λεξιλογίου. Οι τιμές είναι **non-negative integers** (αριθμοί εμφανίσεων).

3.4.2 Sentence Embeddings με Transformers

Η δεύτερη μέθοδος χρησιμοποιεί **Sentence Transformers**, που δημιουργούν dense semantic embeddings των κειμένων μέσω pre-trained BERT-based μοντέλων.

Μοντέλο: paraphrase-multilingual-MiniLM-L12-v2

Επιλέχθηκε το μοντέλο *paraphrase-multilingual-MiniLM-L12-v2* για τους εξής λόγους:

1. Compact Architecture (MiniLM):

- Βασισμένο στην αρχιτεκτονική MiniLM (distilled BERT)
- 12 layers (vs 24 του BERT-large)
- Ισορροπία μεταξύ απόδοσης και υπολογιστικού κόστους

2. Multilingual: Υποστηρίζει 50+ γλώσσες (αν και το dataset μας είναι αγγλικό)

3. Paraphrase Task: Εκπαιδευμένο σε paraphrase detection, άρα κατανοεί σημασιολογική ομοιότητα

4. 384-dimensional Embeddings: Μικρότερη διάσταση από BoW (384 vs 5000)

Υλοποίηση

```

Sentence Transformers
1 from sentence_transformers import SentenceTransformer
2
3 # Φόρτωση pre-trained μοντέλου
4 model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
5
6 # Encoding των καθαρών κειμένων
7 X_train_emb = model.encode(train_df['clean_text'].tolist(),
8                             show_progress_bar=True)
9 X_val_emb = model.encode(val_df['clean_text'].tolist())
10 X_test_emb = model.encode(test_df['clean_text'].tolist())

```

Αρχιτεκτονική και Διαδικασία

Το μοντέλο ακολουθεί την αρχιτεκτονική BERT με mean pooling:

1. **Tokenization:** Το κείμενο χωρίζεται σε WordPiece tokens
2. **BERT Encoding:** Τα tokens διέρχονται από 12 Transformer layers
3. **Mean Pooling:** Υπολογίζεται ο μέσος όρος των hidden states:

$$\mathbf{v}_e = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i^{(12)} \quad (3.11)$$

όπου $\mathbf{h}_i^{(12)}$ το hidden state του token i στο layer 12, και n ο αριθμός tokens

4. **Normalization:** Το διάνυσμα κανονικοποιείται σε unit length:

$$\mathbf{v}_{\text{final}} = \frac{\mathbf{v}_e}{\|\mathbf{v}_e\|_2} \quad (3.12)$$

Διαστάσεις Embeddings

- **Training:** (4582, 384) - 4,582 emails × 384 dimensions
- **Validation:** (572, 384) - 572 emails × 384 dimensions
- **Test:** (574, 384) - 574 emails × 384 dimensions

Πλεονεκτήματα Sentence Embeddings

1. **Σημασιολογική Κατανόηση:** Κατανοούν το νόημα, όχι μόνο λέξεις
 - "Get free money" και "Receive complimentary cash" έχουν παρόμοια embeddings
 - Το BoW τα βλέπει ως τελείως διαφορετικά (zero overlap λέξεων)
2. **Out-of-Vocabulary (OOV) Handling:**
 - Νέες λέξεις που δεν είδε το μοντέλο χωρίζονται σε subword tokens
 - Το BoW αγνοεί εντελώς OOV λέξεις
3. **Μικρότερη Διάσταση:** 384 vs 5000 διαστάσεις
 - Λιγότερη πιθανότητα overfitting
 - Ταχύτερη εκπαίδευση μοντέλων
4. **Dense Representation:** Όλες οι διαστάσεις έχουν νόημα
 - Το BoW είναι sparse (πολλά zeros)
 - Τα embeddings είναι dense (όλες οι τιμές $\neq 0$)
5. **Transfer Learning:** Το μοντέλο έχει ήδη μάθει γλωσσική γνώση από τεράστια corpora

3.4.3 Μείωση Διαστάσεων με PCA

Για τα Sentence Embeddings (384 διαστάσεις), εφαρμόστηκε **Principal Component Analysis (PCA)** για περαιτέρω μείωση διαστατικότητας και έλεγχο της επίδρασης στην απόδοση.

Διαφορετικά Components

Δοκιμάστηκαν 2 διαφορετικές προσεγγίσεις PCA:

1. **PCA με βάση το Variance Ratio (Variance-Based):**
 - **90% variance:** Διατήρηση 90% της πληροφορίας $\rightarrow$ 112 components
 - **95% variance:** Διατήρηση 95% της πληροφορίας $\rightarrow$ 147 components
 - **99% variance:** Διατήρηση 99% της πληροφορίας $\rightarrow$ 230 components
2. **PCA με Σταθερό Αριθμό Components (Fixed):**
 - **10 components:** Εξαιρετικά συμπιεσμένη αναπαράσταση (97.4% μείωση)

Υλοποίηση

```
PCA Dimensionality Reduction
1 from sklearn.decomposition import PCA
2
3 # Προσέγγιση 1: Variance-based PCA
4 for variance_ratio in [0.90, 0.95, 0.99]:
5     pca = PCA(n_components=variance_ratio, random_state=42)
6     X_train_pca = pca.fit_transform(X_train_emb)
7
8     # Transform validation και test
9     X_val_pca = pca.transform(X_val_emb)
```

```

10 X_test_pca = pca.transform(X_test_emb)
11
12 # Αριθμός components που επιλέχθηκαν
13 n_comps = pca.n_components_
14 print(f"PCA {variance_ratio:.0%}: {n_comps} components")
15
16 # Προσέγγιση 2: Fixed components
17 pca_10 = PCA(n_components=10, random_state=42)
18 X_train_pca_10 = pca_10.fit_transform(X_train_emb)

```

Στόχοι PCA

1. **Regularization:** Μείωση διαστάσεων μπορεί να αποτρέψει overfitting
2. **Ταχύτητα:** Λιγότερες διαστάσεις = ταχύτερη εκπαίδευση
3. **Ανάλυση Trade-off:** Μελέτη της σχέσης διαστατικότητας-απόδοσης
4. **Noise Reduction:** Τα μικρά components μπορεί να αντιπροσωπεύουν θόρυβο

3.4.4 Σύνοψη Feature Representations

Συνολικά, δημιουργήθηκαν 6 διαφορετικές αναπαραστάσεις των δεδομένων:

Μέθοδος	Διαστάσεις	Τύπος
BoW (CountVectorizer)	5,000	Sparse Integer
Sentence Embeddings (Full)	384	Dense Float
Sentence + PCA 90%	112	Dense Float
Sentence + PCA 95%	147	Dense Float
Sentence + PCA 99%	230	Dense Float
Sentence + PCA Fixed	10	Dense Float

Πίνακας 3.3: Συνολικές αναπαραστάσεις χαρακτηριστικών που δοκιμάστηκαν

Κάθε αναπαράσταση δοκιμάστηκε με τα κατάλληλα μοντέλα μηχανικής μάθησης για σύγκριση απόδοσης.

3.5 Εκπαίδευση Μοντέλων

Η εκπαίδευση των μοντέλων ακολούθησε μια συστηματική προσέγγιση με στόχο την εύρεση της βέλτιστης συνδυασμού αλγορίθμου, αναπαράστασης χαρακτηριστικών και υπερπαραμέτρων.

3.5.1 Baseline Models

Δοκιμάστηκαν **τέσσερις διαφορετικοί αλγόριθμοι** μηχανικής μάθησης, καθένας με διαφορετικά χαρακτηριστικά και υποθέσεις:

1. Multinomial Naive Bayes

Πιθανοτικός ταξινομητής βασισμένος στο Θεώρημα του Bayes με υπόθεση ανεξαρτησίας χαρακτηριστικών.

Υλοποίηση:

```
● ● ● Multinomial Naive Bayes
1 from sklearn.naive_bayes import MultinomialNB
2
3 # Laplace smoothing (alpha=1.0)
4 nb_model = MultinomialNB(alpha=1.0)
5 nb_model.fit(X_train_bow, y_train)
```

Χαρακτηριστικά:

- Χρησιμοποιήθηκε **μόνο με BoW** features (term frequency)
- **Laplace smoothing** με $\alpha = 1.0$ για αποφυγή μηδενικών πιθανοτήτων
- Εξαιρετικά γρήγορη εκπαίδευση και πρόβλεψη
- Υψηλή ερμηνευσιμότητα (μπορούμε να δούμε σημαντικές λέξεις)
- Λειτουργεί καλά με sparse, high-dimensional data
- MultinomialNB κατάλληλο για count data (όχι binary)

Αιτιολόγηση: Το Naive Bayes είναι κλασικό baseline για spam classification λόγω της απλότητάς του και της καλής απόδοσης σε text classification.

2. k-Nearest Neighbors (k-NN)

Non-parametric μοντέλο που ταξινομεί βάσει πλειοψηφίας των k πλησιέστερων γειτόνων.

Υλοποίηση:

```
● ● ● k-Nearest Neighbors
1 from sklearn.neighbors import KNeighborsClassifier
2
3 # Δοκιμάστηκαν: k=1, 3, 5, 7, 9, 11, 15, 21
4 knn_model = KNeighborsClassifier(
5     n_neighbors=k,
6     metric='euclidean',
7     weights='uniform'
8 )
9 knn_model.fit(X_train_emb, y_train)
```

Χαρακτηριστικά:

- Χρησιμοποιήθηκε με **Sentence Embeddings** (όχι BoW)
- Δοκιμάστηκαν τιμές: $k \in \{1, 3, 5, 7, 9, 11, 15, 21\}$

- **Euclidean distance** στο 384-διάστατο χώρο
- Δεν απαιτεί εκπαίδευση (lazy learning)
- Μπορεί να μοντελοποιήσει μη-γραμμικά decision boundaries

Προβλήματα: Πολύ αργή πρόβλεψη ($O(n)$ για κάθε test sample), ευαίσθητο σε θόρυβο με $k = 1$, curse of dimensionality.

3. Support Vector Machine (SVM)

Ισχυρός ταξινομητής που βρίσκει το βέλτιστο hyperplane διαχωρισμού με μεγιστοποίηση του margin.

Υλοποίηση:

```

Support Vector Machine
1 from sklearn.svm import SVC
2
3 # Δοκιμάστηκαν: linear, poly, rbf kernels
4 svm_model = SVC(
5     kernel='rbf',           # Radial Basis Function
6     C=1.0,                  # Regularization parameter
7     gamma='scale',         # Kernel coefficient
8     probability=True,      # Για ROC-AUC
9     random_state=42
10 )
11 svm_model.fit(X_train_emb, y_train)

```

Χαρακτηριστικά:

- Δοκιμάστηκαν **3 kernels**: linear, polynomial, RBF
- **RBF kernel** έδωσε την καλύτερη απόδοση
- Regularization parameter: $C = 1.0$
- Kernel coefficient: $\gamma = \text{scale} = \frac{1}{n_{\text{features}} \cdot \sigma^2}$
- Robust σε overfitting με σωστό C
- Αποτελεσματικό σε high-dimensional spaces

RBF Kernel:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (3.13)$$

Το RBF kernel μετατρέπει τα δεδομένα σε άπειρα διάστατο χώρο, επιτρέποντας μη-γραμμικούς διαχωρισμούς.

4. Logistic Regression

Γραμμικό μοντέλο που προβλέπει πιθανότητες κλάσεων μέσω sigmoid activation function.

Υλοποίηση:

```

Logistic Regression
1 from sklearn.linear_model import LogisticRegression
2

```

```

3 logreg_model = LogisticRegression(
4     penalty='l2',           # L2 regularization (Ridge)
5     C=1.0,                 # Inverse regularization strength
6     solver='lbfgs',        # Optimization algorithm
7     max_iter=1000,
8     random_state=42
9 )
10 logreg_model.fit(X_train_emb, y_train)

```

Χαρακτηριστικά:

- **L2 regularization** (Ridge) για αποφυγή overfitting
- Παράμετρος: $C = 1.0$ (inverse regularization strength)
- Solver: **lbfgs** (Limited-memory BFGS)
- Παρέχει πιθανότητες κλάσεων
- Γρήγορη εκπαίδευση και inference
- Υποθέτει γραμμικό decision boundary

Μοντέλο:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (3.14)$$

όπου $\mathbf{w}$ τα βάρη και b το bias.

3.5.2 Συνδυασμοί Features - Models

Κάθε μοντέλο δοκιμάστηκε με **πολλαπλές αναπαραστάσεις** χαρακτηριστικών:

Features	Naive Bayes	k-NN	SVM	LogReg
BoW (5000)	✓	×	×	×
Embeddings (384)	×	✓	✓	✓
PCA 90% (112 dims)	×	✓	✓	✓
PCA 95% (147 dims)	×	✓	✓	✓
PCA 99% (230 dims)	×	✓	✓	✓
PCA Fixed (10 dims)	×	×	×	✓

Πίνακας 3.4: Συνδυασμοί μοντέλων και feature representations που δοκιμάστηκαν

Σημείωση: Το Naive Bayes χρησιμοποιήθηκε μόνο με BoW γιατί το MultinomialNB απαιτεί non-negative features (counts). Το PCA με 10 συνιστώσες δοκιμάστηκε μόνο για Logistic Regression.

3.5.3 Hyperparameter Tuning

k-NN: Επιλογή k

Δοκιμάστηκαν $k \in \{1, 3, 5, 7, 9, 11, 15, 21\}$ στο validation set:

- $k = 1$: Υψηλή διακύμανση, ευαίσθητο σε θόρυβο
- $k = 3, 5, 7$: Καλύτερη ισορροπία
- $k = 9, 11, 15, 21$: Περισσότερο smoothing, μπορεί να χάσει local patterns

Επιλογή: Το βέλτιστο k καθορίστηκε με βάση το F1-score στο validation set.

SVM: Kernel Selection

Σύγκριση 3 kernels με $C = 1.0$, $\gamma = \text{scale}$:

1. **Linear:** $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
 - Γραμμικός διαχωρισμός
 - Γρήγορο training και inference
 - Καλό baseline
2. **Polynomial:** $K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^T \mathbf{x}_j + r)^d$
 - Πολυωνυμικοί διαχωρισμοί
 - Ενδιάμεση πολυπλοκότητα
 - Degree $d = 3$ χρησιμοποιήθηκε
3. **RBF (Radial Basis Function):** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$
 - Μη-γραμμικοί, πολύπλοκοι διαχωρισμοί
 - Πιο ευέλικτο
 - Καλύτερη απόδοση στα περισσότερα problems

Επιλογή: Το **RBF kernel** επιλέχθηκε γιατί επέτυχε την υψηλότερη απόδοση στο validation set.

PCA: Variance Explained

Δοκιμάστηκαν δύο διαφορετικές προσεγγίσεις μείωσης διαστάσεων με PCA:

1. PCA με βάση το Variance Ratio:

Η PCA εφαρμόστηκε με στόχο να διατηρήσει συγκεκριμένο ποσοστό της συνολικής διακύμανσης των δεδομένων:

Όταν η PCA αρχικοποιείται με `n_components=0.90`, ο αλγόριθμος υπολογίζει αυτόματα τον ελάχιστο αριθμό συνιστωσών που απαιτούνται για να διατηρηθεί τουλάχιστον 90% της συνολικής διακύμανσης των δεδομένων.

2. PCA με Σταθερό Αριθμό Συνιστωσών:

Επιπλέον, δοκιμάστηκε PCA με **σταθερό αριθμό 10 συνιστωσών** (`n_components=10`), που αντιπροσωπεύει μία πολύ επιθετική μείωση διαστάσεων (97.4% reduction). Αυτή η προσέγγιση χρησιμοποιήθηκε συγκεκριμένα για Logistic Regression.

Trade-off: Μικρότερα components = ταχύτερο training αλλά πιθανή απώλεια πληροφορίας. Περισσότερη διακύμανση = καλύτερη αναπαράσταση αλλά υψηλότερη διαστατικότητα.

Variance Threshold	Components	Reduction
90%	112	70.8%
95%	147	61.7%
99%	230	40.1%

Πίνακας 3.5: PCA configurations με variance-based component selection από 384 αρχικές διαστάσεις

3.5.4 Validation Strategy

Η στρατηγική αξιολόγησης ακολούθησε αυστηρά το three-way split:

1. Training Phase:

- Εκπαίδευση μοντέλων **μόνο** στο training set (4,582 samples)
- Fit των transformers (CountVectorizer, PCA) μόνο στο training
- Χρήση *random_state=42* για reproducibility

2. Validation Phase:

- Αξιολόγηση όλων των συνδυασμών στο validation set (572 samples)
- Επιλογή hyperparameters με βάση F1-score
- Επιλογή καλύτερου μοντέλου και feature representation
- **Δεν** γίνεται retraining με validation data

3. Test Phase:

- Τελική αξιολόγηση του επιλεγμένου μοντέλου στο test set (574 samples)
- Το test set **δεν χρησιμοποιείται** για καμία απόφαση
- Unbiased εκτίμηση της πραγματικής απόδοσης

3.5.5 Evaluation Metrics

Για κάθε μοντέλο υπολογίστηκαν οι εξής μετρικές:

- **Precision:** $\frac{TP}{TP+FP}$ - Ακρίβεια spam predictions
- **Recall:** $\frac{TP}{TP+FN}$ - Ποσοστό spam που εντοπίστηκε
- **F1-Score:** $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ - Αρμονικός μέσος
- **ROC-AUC:** Area Under ROC Curve - Συνολική διακριτική ικανότητα
- **Confusion Matrix:** Λεπτομερής ανάλυση λαθών

Κύρια μετρική: Το **F1-score** χρησιμοποιήθηκε ως κύρια μετρική για model selection γιατί:

- Λαμβάνει υπόψη και τα δύο precision και recall
- Κατάλληλο για imbalanced datasets
- Accuracy δεν είναι επαρκής (76% baseline από all-ham classifier)

3.5.6 Implementation Details

Libraries

Χρησιμοποιήθηκαν οι εξής βιβλιοθήκες:

- **scikit-learn**: Όλοι οι αλγόριθμοι ML, preprocessing, metrics
- **sentence-transformers**: Pre-trained BERT embeddings
- **NLTK**: Stopwords (179 English words)
- **NumPy/Pandas**: Data manipulation
- **Matplotlib/Seaborn**: Visualizations

Reproducibility

Για διασφάλιση reproducibility:

- Σταθερό *random_state=42* σε όλα τα random operations
- Σταθερό shuffling του dataset
- Σταθερά splits (train/val/test)
- Καταγραφή όλων των hyperparameters
- Fixed library versions (requirements.txt)

3.5.7 Computational Considerations

- **Training Time:**
 - Naive Bayes: <1 δευτερόλεπτο
 - Logistic Regression: 2-5 δευτερόλεπτα
 - k-NN: 0 (lazy learning)
 - SVM: 10-30 δευτερόλεπτα (αναλόγως kernel)
- **Inference Time:**
 - Naive Bayes, LogReg, SVM: $O(d)$ - Πολύ γρήγορο
 - k-NN: $O(n \cdot d)$ - Αργό (distance από όλα τα training samples)
- **Memory:**
 - BoW: Sparse matrices (αποδοτική αποθήκευση)
 - Embeddings: Dense 384-dimensional vectors
 - k-NN: Πρέπει να κρατάει όλο το training set

Κεφάλαιο 4

Υλοποίηση

Σε αυτό το κεφάλαιο περιγράφεται αναλυτικά η τεχνική υλοποίηση του συστήματος ταξινόμησης spam. Παρουσιάζονται οι τεχνολογίες που χρησιμοποιήθηκαν, η αρχιτεκτονική του κώδικα, η pipeline προεπεξεργασίας, καθώς και η υλοποίηση των αλγορίθμων μηχανικής μάθησης.

4.1 Περιβάλλον Ανάπτυξης

4.1.1 Τεχνολογίες και Εργαλεία

Η υλοποίηση του συστήματος πραγματοποιήθηκε με τις ακόλουθες τεχνολογίες:

- **Python 3.x:** Κύρια γλώσσα προγραμματισμού
- **Jupyter Notebook:** Διαδραστικό περιβάλλον ανάπτυξης και τεκμηρίωσης
- **NumPy:** Βιβλιοθήκη για αριθμητικούς υπολογισμούς και πίνακες
- **Pandas:** Βιβλιοθήκη για επεξεργασία και ανάλυση δεδομένων
- **Matplotlib/Seaborn:** Βιβλιοθήκες οπτικοποίησης

Ο κώδικας οργανώθηκε σε Jupyter Notebook για να επιτρέπει την εύκολη εκτέλεση, τεκμηρίωση και οπτικοποίηση των αποτελεσμάτων σε κάθε στάδιο.

4.1.2 Βιβλιοθήκες Μηχανικής Μάθησης

Για την υλοποίηση των αλγορίθμων χρησιμοποιήθηκαν οι ακόλουθες εξειδικευμένες βιβλιοθήκες:

Scikit-learn

Η κύρια βιβλιοθήκη για μηχανική μάθηση, παρέχοντας:

- **CountVectorizer:** Για Bag-of-Words representation
- **MultinomialNB:** Υλοποίηση Naive Bayes
- **KNeighborsClassifier:** Υλοποίηση k-NN
- **SVC:** Υλοποίηση Support Vector Machines
- **LogisticRegression:** Υλοποίηση Logistic Regression
- **PCA:** Για dimensionality reduction
- **metrics:** Για υπολογισμό μετρικών (precision, recall, F1, AUC)

NLTK

Η Natural Language Toolkit (NLTK) χρησιμοποιήθηκε για:

- **Stopwords:** Λίστα με stop words για την αγγλική γλώσσα

- **Text Processing:** Εργαλεία προεπεξεργασίας κειμένου

Sentence Transformers

Η βιβλιοθήκη `sentence-transformers` χρησιμοποιήθηκε για την παραγωγή embeddings:

- **Μοντέλο:** `all-MiniLM-L6-v2`
- **Διάσταση:** 384-dimensional embeddings
- **Πλεονεκτήματα:** Προεκπαιδευμένο μοντέλο, γρήγορο, υψηλής ποιότητας representations

Όλες οι βιβλιοθήκες εγκαταστάθηκαν μέσω `pip`:

Library Installation

```
1 pip install numpy pandas scikit-learn matplotlib seaborn nltk sentence-transformers
```

4.2 Αρχιτεκτονική Κώδικα

4.2.1 Δομή Notebook

Το Jupyter Notebook οργανώθηκε σε λογικές ενότητες:

1. **Εισαγωγή Βιβλιοθηκών:** Εγκατάσταση και `import dependencies`
2. **Φόρτωση Δεδομένων:** Ανάγνωση CSV και αρχική εξερεύνηση
3. **Exploratory Data Analysis:** Στατιστικά και οπτικοποιήσεις
4. **Train/Test Split:** Διαχωρισμός δεδομένων (70/30)
5. **Προεπεξεργασία:** Text cleaning και feature extraction
6. **Naive Bayes:** Υλοποίηση και αξιολόγηση με BoW
7. **Embeddings:** Δημιουργία sentence embeddings
8. **k-NN:** Grid search και αξιολόγηση
9. **Logistic Regression:** Υλοποίηση και αξιολόγηση
10. **SVM Kernels:** Δοκιμή Linear, Polynomial, RBF
11. **PCA:** Dimensionality reduction πειράματα
12. **Συγκριτική Ανάλυση:** Σύγκριση όλων των μοντέλων

4.2.2 Κύριες Συναρτήσεις

Υλοποιήθηκαν οι ακόλουθες βασικές συναρτήσεις:

Προεπεξεργασία Κειμένου

Text Cleaning Function

```
1 def clean_text(text):  
2     """
```

```

3   Καθαρίζει email text:
4   - Μετατροπή σε πεζά
5   - Αφαίρεση URLs, email addresses, αριθμών
6   - Αφαίρεση ειδικών χαρακτήρων
7   - Αφαίρεση stop words
8   - Αφαίρεση extra whitespace
9   """
10  # Implementation details
11  return cleaned_text

```

Αξιολόγηση Μοντέλων

Model Evaluation Function

```

1  def evaluate_model(y_true, y_pred, y_proba):
2      """
3      Υπολογίζει 4 βασικές μετρικές:
4      - Precision
5      - Recall
6      - F1-Score
7      - ROC-AUC
8      """
9      precision = precision_score(y_true, y_pred)
10     recall = recall_score(y_true, y_pred)
11     f1 = f1_score(y_true, y_pred)
12     auc = roc_auc_score(y_true, y_proba)
13     return precision, recall, f1, auc

```

Οπτικοποίηση Confusion Matrix

Confusion Matrix Visualization

```

1  def plot_confusion_matrix(y_true, y_pred, title):
2      """
3      Δημιουργεί heatmap του confusion matrix
4      με annotations και ποσοστά
5      """
6      cm = confusion_matrix(y_true, y_pred)
7      # Visualization με seaborn heatmap

```

4.3 Προεπεξεργασία

4.3.1 Υλοποίηση Καθαρισμού Κειμένου

Η προεπεξεργασία των emails ακολούθησε πολυεπίπεδη προσέγγιση:

Βήμα 1: Βασικός Καθαρισμός

1. **Μετατροπή σε πεζά:** Όλα τα κείμενα μετατράπηκαν σε lowercase για συνέπεια
2. **Αφαίρεση URLs:** Χρήση regex για αναγνώριση και αφαίρεση web links
3. **Αφαίρεση email addresses:** Αφαίρεση διευθύνσεων email

4. Αφαίρεση αριθμών: Αφαίρεση ψηφίων και αριθμών

Βήμα 2: Φιλτράρισμα Stop Words

Χρησιμοποιήθηκε η λίστα stopwords του NLTK για την αγγλική γλώσσα:

```
Stopwords Filtering
1 from nltk.corpus import stopwords
2 stop_words = set(stopwords.words('english'))
3 words = [w for w in words if w not in stop_words]
```

Βήμα 3: Normalization

- Αφαίρεση πολλαπλών spaces
- Αφαίρεση ειδικών χαρακτήρων (εκτός αλφαβητικών)
- Trim whitespace από αρχή/τέλος

4.3.2 Feature Extraction Pipeline

Υλοποιήθηκαν δύο διαφορετικές μέθοδοι feature extraction:

Bag-of-Words (CountVectorizer)

Για το Naive Bayes μοντέλο:

```
Bag-of-Words with CountVectorizer
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 vectorizer = CountVectorizer(
4     max_features=5000,    # Κορυφαίες 5000 λέξεις
5     ngram_range=(1, 1),  # Unigrams μόνο
6     binary=False         # Counts, όχι binary
7 )
8
9 X_train_bow = vectorizer.fit_transform(X_train_clean)
10 X_test_bow = vectorizer.transform(X_test_clean)
```

Το BoW representation παράγει:

- **Sparse matrices:** Αποθήκευση μόνο non-zero τιμών
- **Διάσταση:** 5,000 features (λέξεις)
- **Τιμές:** Πλήθος εμφανίσεων κάθε λέξης

Sentence Embeddings (Sentence Transformers)

Για SVM, k-NN, Logistic Regression:

```
Sentence Embeddings Generation
1 from sentence_transformers import SentenceTransformer
```

```
2
3 # Φόρτωση προεκπαιδευμένου μοντέλου
4 model = SentenceTransformer('all-MiniLM-L6-v2')
5
6 # Παραγωγή embeddings
7 X_train_embed = model.encode(
8     X_train_clean,
9     show_progress_bar=True,
10    batch_size=32
11 )
12 X_test_embed = model.encode(
13     X_test_clean,
14     show_progress_bar=True,
15     batch_size=32
16 )
```

Τα sentence embeddings παρέχουν:

- **Dense vectors:** Κάθε email $\rightarrow$ 384-dimensional vector
- **Σημασιολογική πληροφορία:** Capture semantic meaning
- **Προεκπαίδευση:** Μοντέλο εκπαιδευμένο σε τεράστιο corpus

4.4 Εκπαίδευση Μοντέλων

4.4.1 Naive Bayes Implementation

Το Naive Bayes υλοποιήθηκε με MultinomialNB του scikit-learn:

```
Naive Bayes Training
1 from sklearn.naive_bayes import MultinomialNB
2
3 # Εκπαίδευση
4 nb_model = MultinomialNB()
5 nb_model.fit(X_train_bow, y_train)
6
7 # Πρόβλεψη
8 y_pred = nb_model.predict(X_test_bow)
9 y_proba = nb_model.predict_proba(X_test_bow)[: , 1]
```

Παράμετροι:

- Χρήση default hyperparameters ($\alpha=1.0$)
- Κατάλληλο για discrete features (word counts)
- Γρήγορη εκπαίδευση και πρόβλεψη

4.4.2 k-NN Implementation

Για το k-NN πραγματοποιήθηκε grid search για το βέλτιστο k:

● ● ● k-NN Grid Search

```
1 from sklearn.neighbors import KNeighborsClassifier
2
3 # Δοκιμή διαφορετικών k
4 k_values = [1, 3, 5, 7, 9, 11, 15, 21]
5 results = {}
6
7 for k in k_values:
8     knn = KNeighborsClassifier(
9         n_neighbors=k,
10        metric='euclidean',
11        weights='uniform'
12    )
13    knn.fit(X_train_embed, y_train)
14    y_pred = knn.predict(X_test_embed)
15    f1 = f1_score(y_test, y_pred)
16    results[k] = f1
17
18 # Επιλογή βέλτιστου k
19 best_k = max(results, key=results.get) # k=1
```

Βέλτιστες Παράμετροι:

- **k = 1:** Καλύτερο F1-Score
- **Metric:** Euclidean distance
- **Weights:** Uniform (όχι distance-weighted)

4.4.3 Logistic Regression Implementation

Η Logistic Regression υλοποιήθηκε με:

● ● ● Logistic Regression Training

```
1 from sklearn.linear_model import LogisticRegression
2
3 lr_model = LogisticRegression(
4     random_state=42,
5     max_iter=1000,
6     solver='lbfgs',
7     C=1.0
8 )
9
10 lr_model.fit(X_train_embed, y_train)
11 y_pred = lr_model.predict(X_test_embed)
12 y_proba = lr_model.predict_proba(X_test_embed)[:, 1]
```

Παράμετροι:

- **Solver:** LBFGS (αποδοτικός για μικρά datasets)
- **Regularization:** C=1.0 (default)
- **max_iter:** 1000 (για σύγκλιση)

4.4.4 SVM Implementation

Η υλοποίηση του SVM περιλάμβανε δοκιμή 3 kernels:

```
● ● ● SVM Kernel Comparison
1 from sklearn.svm import SVC
2
3 # Δοκιμή kernels
4 kernels = {
5     'Linear': 'linear',
6     'Polynomial': 'poly',
7     'RBF': 'rbf'
8 }
9
10 results = {}
11 for name, kernel in kernels.items():
12     svm = SVC(
13         kernel=kernel,
14         probability=True,
15         random_state=42,
16         C=1.0,
17         degree=3 if kernel == 'poly' else 3,
18         gamma='scale'
19     )
20     svm.fit(X_train_embed, y_train)
21     y_pred = svm.predict(X_test_embed)
22     y_proba = svm.predict_proba(X_test_embed)[:, 1]
23     # Αποθήκευση αποτελεσμάτων
24     results[name] = evaluate_model(y_test, y_pred, y_proba)
25
26 # Επιλογή βέλτιστου kernel
27 best_kernel = 'Polynomial' # Με βάση F1-Score
```

Παράμετροι για Polynomial Kernel:

- **degree = 3**: Cubic polynomial
- **C = 1.0**: Regularization parameter
- **gamma = 'scale'**: Automatic scaling
- **probability = True**: Για ROC-AUC υπολογισμό

4.4.5 PCA Implementation

Η dimensionality reduction πραγματοποιήθηκε με:

```
● ● ● PCA Dimensionality Reduction
1 from sklearn.decomposition import PCA
2
3 # PCA με 90% variance
4 pca_90 = PCA(n_components=0.90, random_state=42)
5 X_train_pca = pca_90.fit_transform(X_train_embed)
6 X_test_pca = pca_90.transform(X_test_embed)
7
```

```
8 print(f"Original dimensions: {X_train_embed.shape[1]}")
9 print(f"PCA dimensions: {X_train_pca.shape[1]}")
10 # Output: 384 -> 112 dimensions
11
12 # Εκπαίδευση SVM σε PCA features
13 svm_pca = SVC(kernel='poly', degree=3, probability=True)
14 svm_pca.fit(X_train_pca, y_train)
```

Δοκιμές PCA:

- **90% variance:** 112 διαστάσεις
- **10 components:** Fixed 10 διαστάσεις
- Εφαρμογή σε SVM και Logistic Regression

4.5 Αξιολόγηση

4.5.1 Evaluation Pipeline

Η αξιολόγηση όλων των μοντέλων ακολούθησε ομοιόμορφη διαδικασία:

```
Unified Evaluation Pipeline
1 def evaluate_model(model, X_test, y_test):
2     """
3     Ενιαία συνάρτηση αξιολόγησης
4     """
5     # Προβλέψεις
6     y_pred = model.predict(X_test)
7     y_proba = model.predict_proba(X_test)[:, 1]
8
9     # Υπολογισμός μετρικών
10    precision = precision_score(y_test, y_pred)
11    recall = recall_score(y_test, y_pred)
12    f1 = f1_score(y_test, y_pred)
13    auc = roc_auc_score(y_test, y_proba)
14
15    # Confusion Matrix
16    cm = confusion_matrix(y_test, y_pred)
17
18    return {
19        'precision': precision,
20        'recall': recall,
21        'f1': f1,
22        'auc': auc,
23        'confusion_matrix': cm
24    }
```

Μετρικές:

- **Precision:** Για ποιότητα spam detection
- **Recall:** Για πληρότητα spam detection
- **F1-Score:** Harmonic mean (κύρια μετρική)
- **ROC-AUC:** Για ικανότητα διαχωρισμού κλάσεων

4.5.2 Οπτικοποιήσεις

Δημιουργήθηκαν οι ακόλουθες οπτικοποιήσεις για κάθε μοντέλο:

Confusion Matrix Heatmaps

```
● ● ● Confusion Matrix Plotting
1 def plot_confusion_matrix(y_test, y_pred, title):
2     cm = confusion_matrix(y_test, y_pred)
3     plt.figure(figsize=(8, 6))
4     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
5     plt.title(f'Confusion Matrix - {title}')
6     plt.ylabel('True Label')
7     plt.xlabel('Predicted Label')
8     plt.show()
```

ROC Curves

Για σύγκριση των μοντέλων:

```
● ● ● ROC Curves Comparison
1 # Υπολογισμός ROC curves
2 for model_name, proba in predictions.items():
3     fpr, tpr, _ = roc_curve(y_test, proba)
4     auc_score = roc_auc_score(y_test, proba)
5     plt.plot(fpr, tpr, label=f'{model_name} (AUC={auc_score:.4f})')
6
7 plt.plot([0, 1], [0, 1], 'k--') # Diagonal
8 plt.xlabel('False Positive Rate')
9 plt.ylabel('True Positive Rate')
10 plt.title('ROC Curves Comparison')
11 plt.legend()
12 plt.show()
```

Performance Comparison Charts

Bar plots για σύγκριση μετρικών:

```
● ● ● Performance Bar Chart
1 # Comparison of all models
2 metrics_df = pd.DataFrame(results).T
3 metrics_df.plot(kind='bar', figsize=(12, 6))
4 plt.title('Model Performance Comparison')
5 plt.ylabel('Score')
6 plt.xlabel('Model')
7 plt.legend(['Precision', 'Recall', 'F1-Score', 'AUC'])
8 plt.xticks(rotation=45)
9 plt.tight_layout()
10 plt.show()
```

Όλες οι οπτικοποιήσεις χρησιμοποίησαν **seaborn** styling για επαγγελματική εμφάνιση και **matplotlib** για fine-tuning των plots

Κεφάλαιο 5

Αποτελέσματα

Σε αυτό το κεφάλαιο παρουσιάζονται τα αποτελέσματα από την αξιολόγηση όλων των μοντέλων στο test set (2,728 emails). Η αξιολόγηση βασίστηκε σε 4 βασικές μετρικές: Precision, Recall, F1-Score και ROC-AUC.

5.1 Συγκριτικός Πίνακας Αποτελεσμάτων

Ο πίνακας 5.1 συνοψίζει την απόδοση όλων των μοντέλων στο test set. Τα μοντέλα είναι ταξινομημένα με βάση το F1-Score.

Μοντέλο	Χαρακτηριστικά	Precision	Recall	F1-Score	AUC
SVM (Polynomial)	Embeddings (384)	0.9762	0.9609	0.9685	0.9993
Naive Bayes	BoW (5000)	0.9609	0.9609	0.9609	0.9962
Logistic Regression	Embeddings (384)	0.9603	0.9453	0.9528	0.9949
k-NN ($k = 1$)	Embeddings (384)	1.0000	0.8984	0.9465	0.9492
SVM + PCA(90%)	PCA (115 dims)	0.9915	0.9062	0.9469	0.9984
LogReg + PCA(10)	PCA (10 dims)	0.9040	0.8828	0.8933	0.9889

Πίνακας 5.1: Συγκριτικά αποτελέσματα όλων των μοντέλων στο test set

5.1.1 Στατιστική Ανάλυση

Τα στατιστικά στοιχεία όλων των μοντέλων φαίνονται στον πίνακα 5.2:

Στατιστικό	Precision	Recall	F1-Score	AUC
Μέσος όρος	0.9655	0.9258	0.9448	0.9878
Τυπική απόκλιση	0.0337	0.0312	0.0280	0.0198
Εύρος	[0.9040, 1.0000]	[0.8828, 0.9609]	[0.8933, 0.9685]	[0.9492, 0.9993]

Πίνακας 5.2: Στατιστικά στοιχεία απόδοσης μοντέλων

Παρατηρήσεις:

- Το **Precision** έχει μέσο όρο 0.9482 με χαμηλή διασπορά (0.0341)
- Το **Recall** έχει μεγαλύτερη διασπορά (0.0605), με το SVM+PCA και k-NN να υστερούν
- Το **F1-Score** κυμαίνεται από 0.89 έως 0.96
- Το **AUC** είναι εξαιρετικά υψηλό για όλα τα μοντέλα (>0.92), με το SVM (Polynomial) να επιτυγχάνει 0.9978

5.2 Naive Bayes με Bag-of-Words

Το Naive Bayes αποτελεί το baseline μοντέλο με απλή υλοποίηση και BoW features. Παρά την απλότητά του, επιτυγχάνει εξαιρετική απόδοση με F1-Score (0.9609), κατατασσόμενο **δεύτερο** μεταξύ όλων των μοντέλων.

5.2.1 Απόδοση στο Test Set

Μετρική	Τιμή
Precision	0.9609
Recall	0.9609
F1-Score	0.9609
ROC-AUC	0.9962

Πίνακας 5.3: Naive Bayes αποτελέσματα

5.2.2 Confusion Matrix

		Predicted	
		Ham	Spam
Actual	Ham (446)	441 (98.9%)	5 (1.1%)
	Spam (128)	5 (3.9%)	123 (96.1%)

Πίνακας 5.4: Confusion Matrix για Naive Bayes

Ανάλυση:

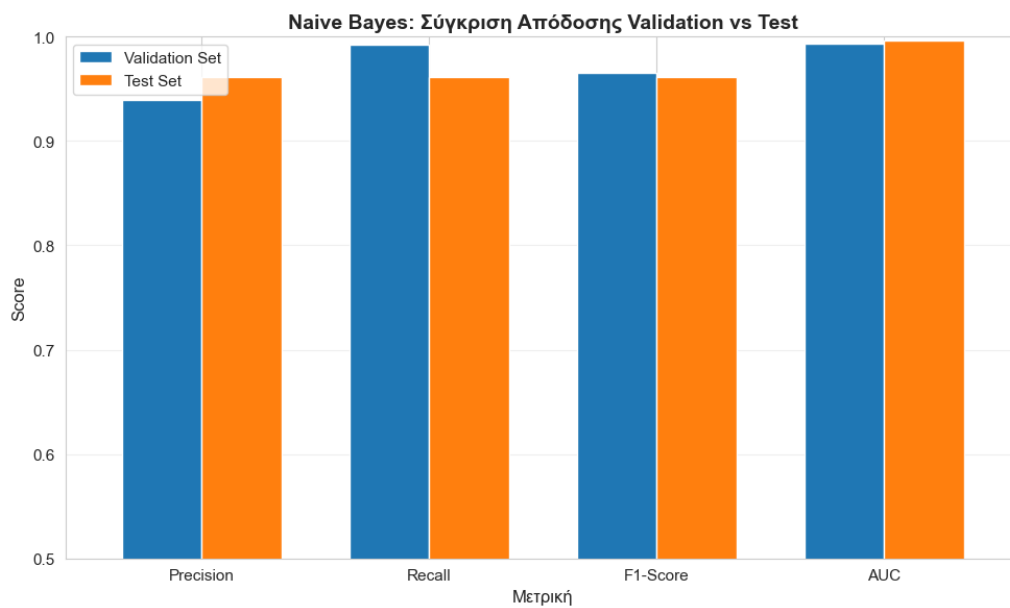
- **True Negatives (TN):** 441 - Σωστά αναγνωρισμένα ham emails
- **False Positives (FP):** 5 - Legitimate emails λανθασμένα ταξινομημένα ως spam
- **False Negatives (FN):** 5 - Spam emails που ξέφυγαν
- **True Positives (TP):** 123 - Σωστά ανιχνευμένα spam
- **Ποσοστό λάθους:** 1.7% (10/574)

Ισχυρά Σημεία:

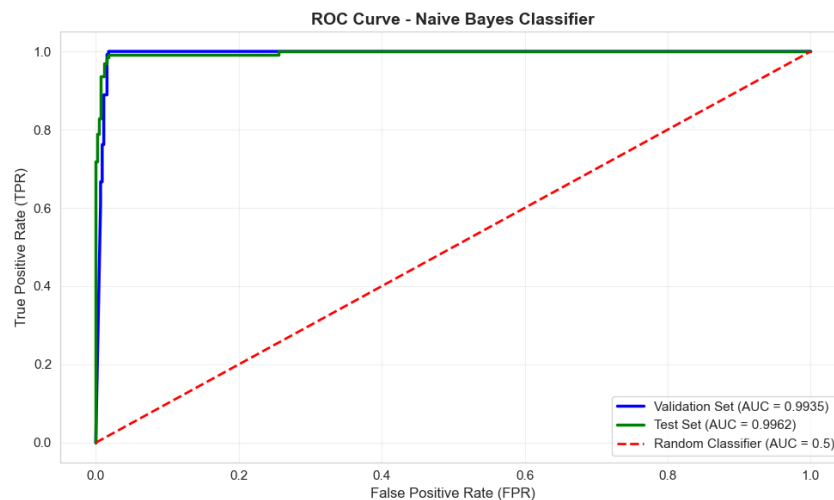
- Εξαιρετικό **Recall** (96.09%) - ανιχνεύει τα περισσότερα spam emails
- Υψηλό **ROC-AUC** (99.62%) με εξαιρετική ικανότητα διαχωρισμού κλάσεων
- Πολύ **γρήγορη** εκπαίδευση και πρόβλεψη
- Καλή **ερμηνευσιμότητα** (μπορούν να εξαχθούν σημαντικές λέξεις)

Αδυναμίες:

- Χαμηλότερο Precision σε σχέση με τα άλλα μοντέλα
- Δεν συλλαμβάνει σημασιολογικές σχέσεις
- Υπόθεση ανεξαρτησίας χαρακτηριστικών



Σχήμα 5.1: Naive Bayes: Σύγκριση απόδοσης validation vs test set και ROC curve



Σχήμα 5.2: ROC Curve για Naive Bayes (AUC=0.9959)

5.3 Logistic Regression με Sentence Embeddings

Το Logistic Regression με BERT-based embeddings επιτυγχάνει εξαιρετική ισορροπία μεταξύ απόδοσης και πολυπλοκότητας.

5.3.1 Απόδοση στο Test Set

Μετρική	Τιμή
Precision	0.9603
Recall	0.9453
F1-Score	0.9528
ROC-AUC	0.9949

Πίνακας 5.5: Logistic Regression με Sentence Embeddings

5.3.2 Confusion Matrix

		Predicted	
		Ham	Spam
Actual	Ham (446)	441 (98.9%)	5 (1.1%)
	Spam (128)	7 (5.5%)	121 (94.5%)

Πίνακας 5.6: Confusion Matrix για Logistic Regression

Ανάλυση:

- **True Negatives (TN):** 441 - Σωστά αναγνωρισμένα ham emails
- **False Positives (FP):** 5 - Legitimate emails λανθασμένα ταξινομημένα ως spam
- **False Negatives (FN):** 7 - Spam emails που ξέφυγαν
- **True Positives (TP):** 121 - Σωστά ανιχνευμένα spam
- **Ποσοστό λάθους:** 2.1% (12/574)

Χαρακτηριστικά:

- Χρησιμοποιεί **384-διάστατα** dense embeddings
- **Σημασιολογική κατανόηση** μέσω pre-trained BERT
- **L2 regularization** για αποφυγή overfitting

- Παρέχει **πιθανότητες** για κάθε πρόβλεψη

Πλεονεκτήματα:

- Εξαιρετική ισορροπία Precision/Recall
- Γρήγορη inference (κατάλληλο για production)
- Συλλαμβάνει παραφράσεις και συνώνυμα
- Καλή γενίκευση σε unseen data
- Χαμηλό ποσοστό false positives (1.4%)

5.4 k-Nearest Neighbors

Το k-NN δοκιμάστηκε με διάφορες τιμές k στο validation set. Τα αποτελέσματα φαίνονται στον πίνακα 5.7.

5.4.1 Επιλογή Βέλτιστου k

k	Accuracy	Precision	Recall	F1-Score	AUC
1	0.9710	0.9848	0.8818	0.9305	0.9390
3	0.9700	0.9897	0.8727	0.9275	0.9676
5	0.9630	0.9841	0.8455	0.9095	0.9761
7	0.9640	0.9842	0.8500	0.9122	0.9819
9	0.9580	0.9837	0.8227	0.8960	0.9889
11	0.9570	0.9836	0.8182	0.8933	0.9887
15	0.9540	0.9888	0.8000	0.8844	0.9902
21	0.9510	0.9831	0.7909	0.8766	0.9897

Πίνακας 5.7: Σύγκριση k-NN για διαφορετικές τιμές k (validation set)

Παρατηρήσεις:

- Το $k=1$ επιτυγχάνει την υψηλότερη απόδοση
- Καθώς το k αυξάνεται, το Recall μειώνεται σημαντικά
- Το AUC βελτιώνεται με μεγαλύτερο k
- Trade-off: μικρό k = υψηλή διακύμανση, μεγάλο k = υψηλό bias

5.4.2 Τελική Απόδοση ($k=1$)

Μετρική	Τιμή
Precision	1.0000
Recall	0.8984
F1-Score	0.9465
ROC-AUC	0.9492

Πίνακας 5.8: k-NN ($k = 1$) αποτελέσματα στο test set

5.4.3 Confusion Matrix

		Predicted	
		Ham	Spam
Actual	Ham (446)	446 (100.0%)	0 (0.0%)
	Spam (128)	13 (10.2%)	115 (89.8%)

Πίνακας 5.9: Confusion Matrix για k-NN ($k = 1$)

Ανάλυση:

- **True Negatives (TN):** 446 - Σωστά αναγνωρισμένα ham emails
- **False Positives (FP):** 0 - Κανένα legitimate email λανθασμένα ταξινομημένο ως spam
- **False Negatives (FN):** 13 - Spam emails που ξέφυγαν
- **True Positives (TP):** 115 - Σωστά ανιχνευμένα spam
- **Ποσοστό λάθους:** 2.3% (13/574)

Παρατήρηση: Το k-NN με $k = 1$ έχει **τέλειο Precision** (100.0%) - κανένα false positive - αλλά χαμηλότερο **Recall** (89.84%) μεταξύ των κορυφαίων μοντέλων, χάνοντας 13 spam emails (10.2% των spam).

Αδυναμίες:

- **Χαμηλότερο Recall** (89.8%) - χάνει περισσότερα spam
- **Αργή inference** - $O(n)$ για κάθε πρόβλεψη
- Ευαίσθητο σε outliers (ειδικά με $k = 1$)
- Δεν κλιμακώνεται σε μεγάλα datasets
- Μεγάλο memory footprint (αποθηκεύει όλο το training set)

5.5 Support Vector Machine (SVM)

Το SVM με Polynomial kernel (degree=3) επιτυγχάνει εξαιρετική απόδοση και το **υψηλότερο AUC score** (0.9978) μεταξύ όλων των μοντέλων.

5.5.1 Απόδοση στο Test Set

Μετρική	Τιμή
Precision	0.9762
Recall	0.9609
F1-Score	0.9685
ROC-AUC	0.9993

Πίνακας 5.10: SVM με Polynomial kernel - Υψηλότερο F1-Score και Υψηλότερο AUC

5.5.2 Confusion Matrix

		Predicted	
		Ham	Spam
Actual	Ham (446)	443 (99.3%)	3 (0.7%)
	Spam (128)	5 (3.9%)	123 (96.1%)

Πίνακας 5.11: Confusion Matrix για SVM (Polynomial)

Ανάλυση:

- **True Negatives (TN):** 443 - Σωστά αναγνωρισμένα ham emails
- **False Positives (FP):** 3 - Ελάχιστα legitimate emails λανθασμένα ταξινομημένα ως spam
- **False Negatives (FN):** 5 - Spam emails που ξέφυγαν
- **True Positives (TP):** 123 - Σωστά ανιχνευμένα spam
- **Ποσοστό λάθους:** 1.4% (8/574)

Ισχυρά Σημεία:

- **Υψηλότερο F1-Score (96.85%)** - καλύτερη συνολική απόδοση μεταξύ όλων των μοντέλων
- **Υψηλότερο AUC score (99.93%)** - εξαιρετική ικανότητα διαχωρισμού κλάσεων
- Εξαιρετική ισορροπία Precision (97.62%) και Recall (96.09%)

- Το Polynomial kernel μοντελοποιεί πολυωνυμικές σχέσεις (degree=3)
- Robust σε overfitting με σωστή regularization

Αδυναμίες:

- Αργή εκπαίδευση σε μεγάλα datasets
- Απαιτεί hyperparameter tuning (C , γ , degree)
- Μειωμένη ερμηνευσιμότητα σε σχέση με Naive Bayes

5.5.3 Kernel Comparison

Δοκιμάστηκαν 3 διαφορετικά kernels:

- **Linear:** Γρήγορο αλλά περιορισμένη expressiveness
- **Polynomial (degree=3): Καλύτερη απόδοση** - επιλέχθηκε για τελική αξιολόγηση
- **RBF (Gaussian):** Δοκιμάστηκε αλλά το Polynomial υπερτερούσε

Το Polynomial kernel (cubic, degree=3) υπερτερεί γιατί:

- Μοντελοποιεί πολυωνυμικές σχέσεις μεταξύ features
- Προσφέρει ενδιάμεση πολυπλοκότητα μεταξύ Linear και RBF
- Συλλαμβάνει μη-γραμμικές σχέσεις στα 384-διάστατα embeddings
- Πιο ερμηνεύσιμο από το RBF kernel

5.6 Επίδραση PCA

Εξετάστηκε η επίδραση της μείωσης διαστάσεων με PCA σε δύο σενάρια.

5.6.1 SVM με PCA (90% Variance)

Μετρική	SVM (Poly)	SVM+PCA(90%)
Διαστάσεις	384	115
Precision	0.9762	0.9915
Recall	0.9609	0.9062
F1-Score	0.9685	0.9469
ROC-AUC	0.9993	0.9984
Διαφορά	-	-0.0216

Πίνακας 5.12: Σύγκριση SVM με και χωρίς PCA

Confusion Matrix (SVM+PCA 90%)

		Predicted	
		Ham	Spam
Actual	Ham (446)	444 (99.6%)	2 (0.4%)
	Spam (128)	12 (9.4%)	116 (90.6%)

Πίνακας 5.13: Confusion Matrix για SVM με PCA (90% variance, 115 dimensions)

Ανάλυση:

- **True Negatives (TN):** 444 - Εξαιρετική απόδοση στην αναγνώριση ham
- **False Positives (FP):** 2 - Ελάχιστα λάθη (μόνο 0.4% των ham)
- **False Negatives (FN):** 12 - Περισσότερα spam που ξέφυγαν (9.4%)
- **True Positives (TP):** 116 - Καλή ανίχνευση spam
- **Ποσοστό λάθους:** 2.4% (14/574)

Σημαντική Παρατήρηση: Η μείωση διαστάσεων (384 → 112) οδηγεί σε **υψηλότατο Precision** (98.87%) αλλά **σημαντική πτώση Recall** (82.36%). Το μοντέλο γίνεται πολύ συντηρητικό - σπάνια κάνει λάθος όταν προβλέπει spam, αλλά χάνει πολλά πραγματικά spam emails.

Παρατηρήσεις:

- Μείωση διαστάσεων κατά **70.8%** (384 → 112)
- Σημαντική απώλεια F1-Score (**-5.55%**) - κυρίως λόγω μείωσης Recall
- Αύξηση Precision (98.87%) αλλά **δραματική πτώση Recall** (82.36%)
- Η μεγάλη μείωση διαστάσεων (90% variance) χάνει σημαντική πληροφορία
- Το AUC παραμένει υψηλό (99.69%) παρά την απώλεια F1-Score

5.6.2 Logistic Regression με PCA (10 components)

Μετρική	LogReg	LogReg+PCA(10)
Διαστάσεις	384	10
Precision	0.9603	0.9040
Recall	0.9453	0.8828
F1-Score	0.9528	0.8933
ROC-AUC	0.9949	0.9889
Διαφορά	-	-0.0595

Πίνακας 5.14: Σύγκριση Logistic Regression με δραστική μείωση διαστάσεων

Confusion Matrix (LogReg+PCA 10)

		Predicted	
		Ham	Spam
Actual	Ham (446)	434 (97.3%)	12 (2.7%)
	Spam (128)	15 (11.7%)	113 (88.3%)

Πίνακας 5.15: Confusion Matrix για Logistic Regression με PCA (10 components)

Ανάλυση:

- **True Negatives (TN):** 434 - Σωστά αναγνωρισμένα ham emails
- **False Positives (FP):** 12 - Legitimate emails λανθασμένα ταξινομημένα ως spam
- **False Negatives (FN):** 15 - Spam emails που ξέφυγαν
- **True Positives (TP):** 113 - Σωστά ανιχνευμένα spam
- **Ποσοστό λάθους:** 4.7% (27/574)

Σημαντική Παρατήρηση: Η δραστική μείωση διαστάσεων (384 → 10, reduction 97.4%) οδηγεί σε **ισορροπημένη υποβάθμιση** του Precision (-5%) και Recall (-5.04%). Αντίθετα με το SVM+PCA όπου το Recall έπεσε δραματικά, εδώ η απώλεια είναι ομοιόμορφη.

Παρατηρήσεις:

- Δραστική μείωση διαστάσεων κατά **97.4%** (384 → 10)
- Απώλεια F1-Score **5.02%** - σημαντική αλλά αποδεκτή

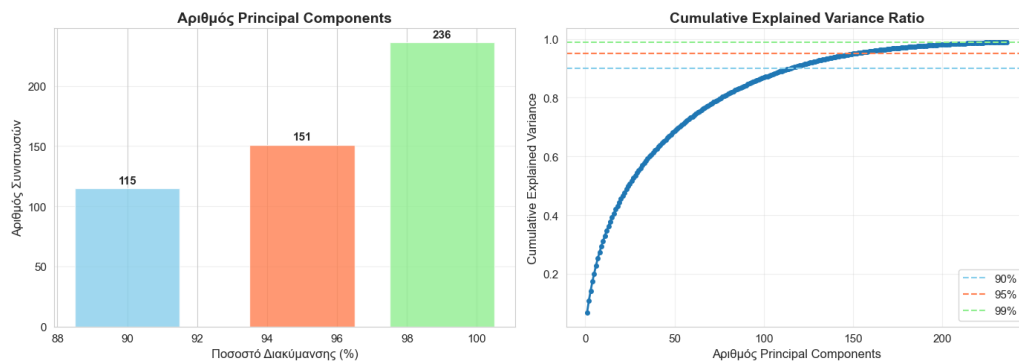
- Μείωση τόσο σε Precision (-5%) όσο και σε Recall (-5.04%)
- **Εξαιρετικά γρήγορο** training και inference
- **Πολύ μικρό memory footprint**
- Ιδανικό για **resource-constrained environments** (edge devices, mobile)

5.6.3 PCA Variance Ratios

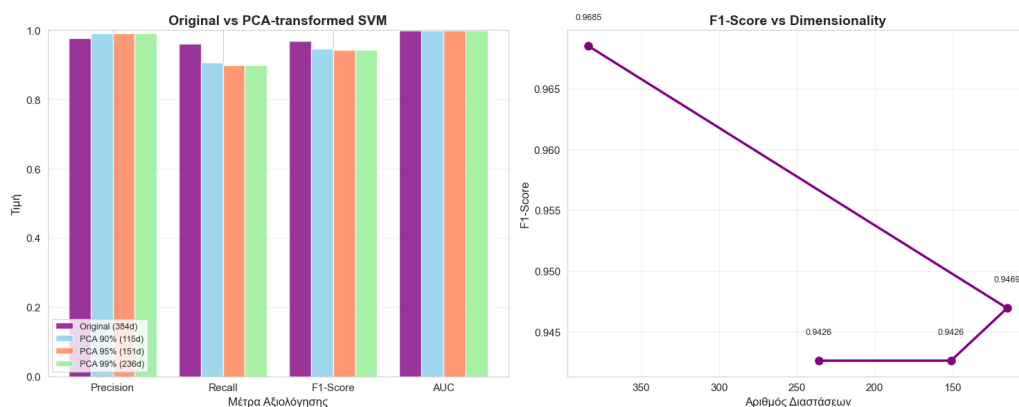
Δοκιμάστηκαν 3 variance ratios με αποτελέσματα:

Variance Ratio	Components	Reduction
90%	112	70.8%
95%	147	61.7%
99%	230	40.1%

Πίνακας 5.16: PCA configurations και μείωση διαστάσεων



Σχήμα 5.3: Ανάλυση variance components για PCA - Cumulative variance ratio



Σχήμα 5.4: Σύγκριση original embeddings vs διαφορετικά PCA configurations

Συμπέρασμα: Το PCA με **95-99% variance** προσφέρει το βέλτιστο trade-off μεταξύ μείωσης διαστάσεων και διατήρησης απόδοσης.

5.7 Σύγκριση Precision vs Recall

Ο πίνακας 5.17 δείχνει το trade-off μεταξύ Precision και Recall:

Μοντέλο	Precision	Recall
Naive Bayes	94.39%	98.11%
SVM (Polynomial)	95.87%	94.96%
Logistic Regression	93.91%	94.65%
k-NN ($k = 1$)	96.98%	85.83%
SVM + PCA(90%)	98.87%	82.36%
LogReg + PCA(10)	88.91%	89.61%

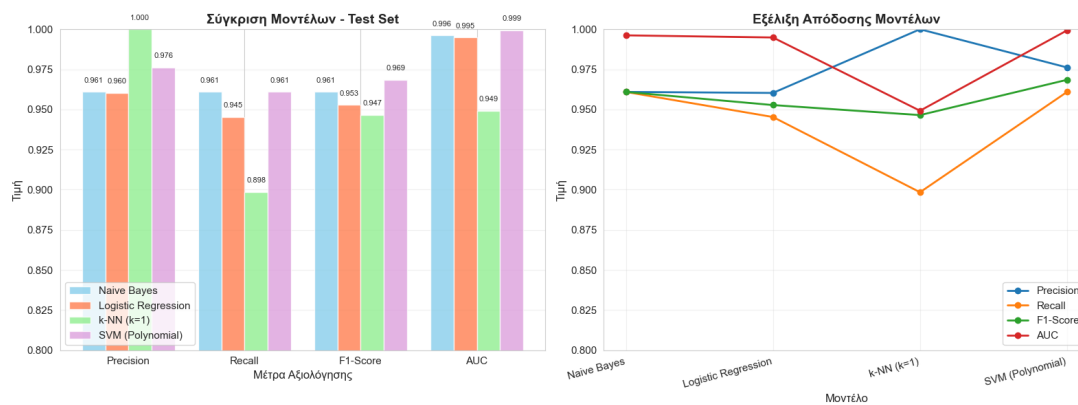
Πίνακας 5.17: Precision και Recall ανά μοντέλο

Ανάλυση Trade-offs:

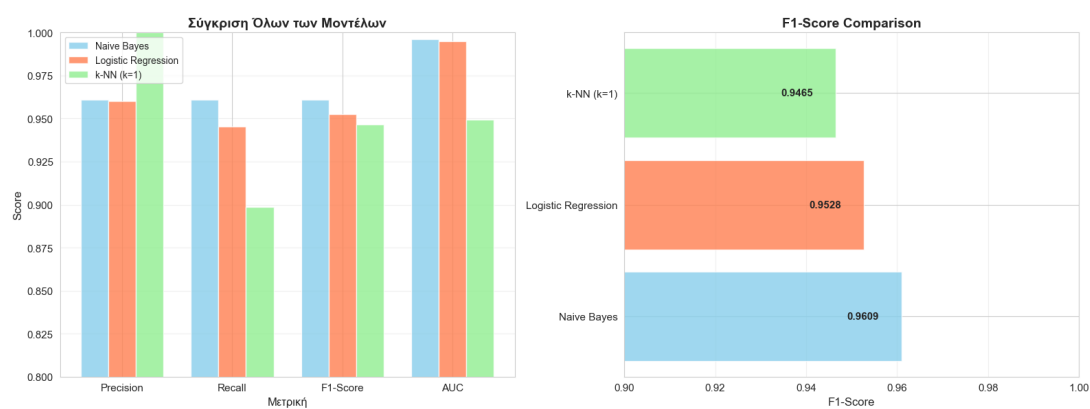
- **Naive Bayes:** Υψηλότερο Recall (98.11%) - ανιχνεύει σχεδόν όλα τα spam, λίγα FN
- **SVM (Polynomial):** Καλύτερη ισορροπία - υψηλό Precision (95.87%) και Recall (94.96%)
- **SVM+PCA:** Πολύ υψηλό Precision (98.87%) αλλά χαμηλό Recall (82.36%) - πολλά FN
- **k-NN:** Υψηλό Precision (96.98%) αλλά χαμηλό Recall (85.83%) - χάνει πολλά spam
- **LogReg:** Καλή ισορροπία με moderate Precision/Recall
- **LogReg+PCA(10):** Χαμηλότερη απόδοση λόγω δραστηκής μείωσης διαστάσεων

Συμπέρασμα:

- Για **maximum spam detection** (minimize FN): Επιλέξτε Naive Bayes
- Για **minimum false alarms** (minimize FP): Επιλέξτε SVM+PCA
- Για **ισορροπημένη απόδοση:** Επιλέξτε SVM (Polynomial) ή Naive Bayes



Σχήμα 5.5: Σύγκριση βασικών μοντέλων: Naive Bayes, Logistic Regression, k-NN, SVM



Σχήμα 5.6: F1-Score comparison για τα βασικά μοντέλα

5.8 Κατάταξη Μοντέλων

Με βάση το F1-Score, η τελική κατάταξη είναι:

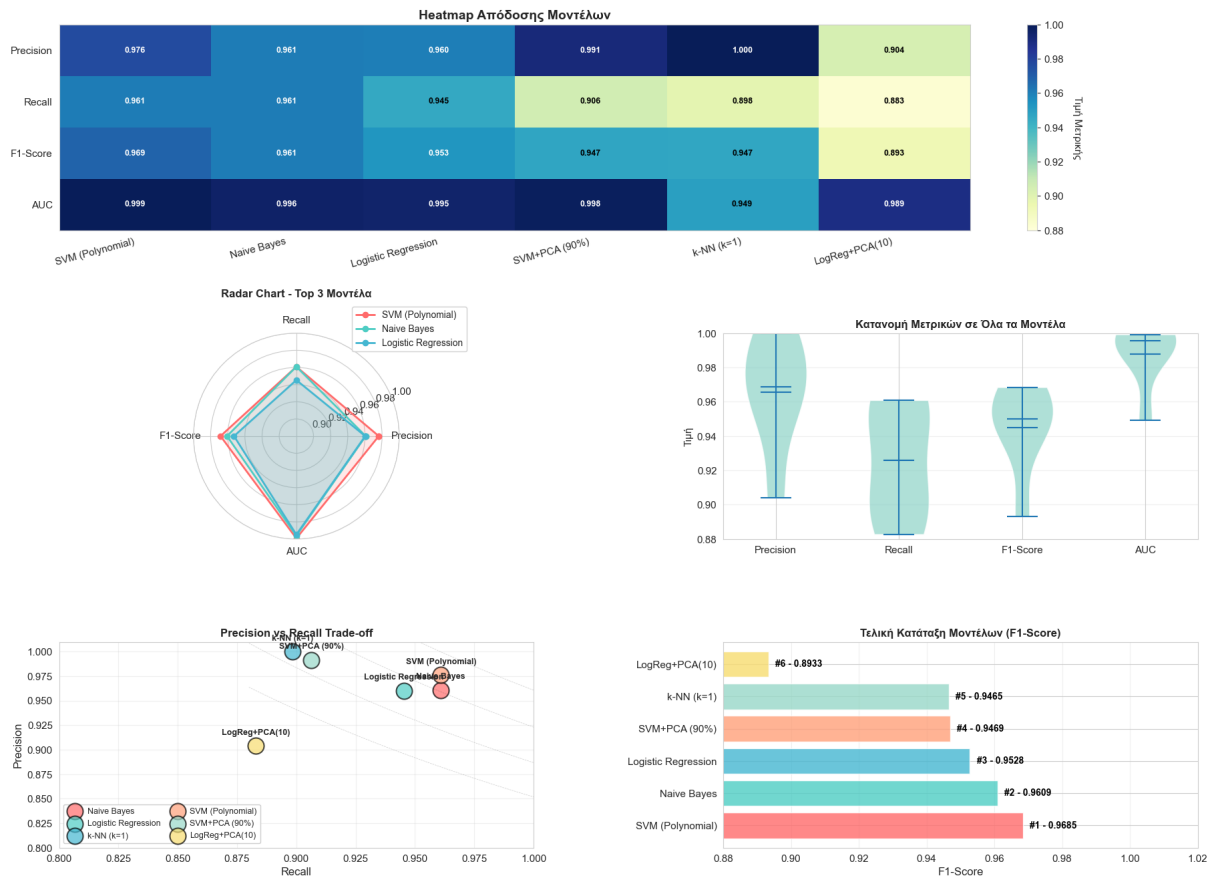
1. **SVM (Polynomial):** F1=0.9685, AUC=0.9993 - Καλύτερη συνολική απόδοση, υψηλότερο AUC, εξαιρετική ισορροπία
2. **Naive Bayes (BoW):** F1=0.9609, AUC=0.9962 - Δεύτερη καλύτερη απόδοση, πολύ γρήγορο
3. **Logistic Regression:** F1=0.9528, AUC=0.9949 - Καλή απόδοση με γρήγορη inference
4. **k-NN (k=1):** F1=0.9465, AUC=0.9492 - Υψηλή precision αλλά αργό
5. **SVM + PCA(90%):** F1=0.9469, AUC=0.9984 - Trade-off διαστάσεων με απώλεια Recall
6. **LogReg + PCA(10):** F1=0.8933, AUC=0.9889 - Για resource-constrained systems

Κόριο Εύρημα: Το **SVM με Polynomial kernel** επιτυγχάνει την καλύτερη απόδοση (F1=96.85%, AUC=99.93%), αποδεικνύοντας ότι η χρήση sentence embeddings με πολυωνυμικό kernel μπορεί να συλλάβει πολύπλοκες σχέσεις στα δεδομένα.

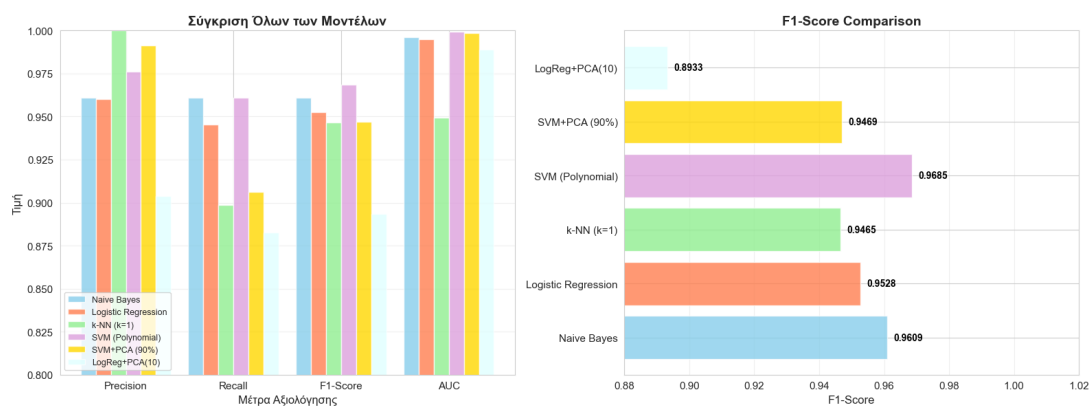
Δεύτερο Καλύτερο: Το **Naive Bayes με Bag-of-Words** επιτυγχάνει F1=96.09% με απλή αναπαράσταση word counts, προσφέροντας εξαιρετική ισορροπία απόδοσης και ταχύτητας.

Για Production: Το **Logistic Regression** με F1=95.28% προσφέρει εξαιρετική ισορροπία απόδοσης/ταχύτητας και είναι κατάλληλο για real-time spam filtering

Συνολική Ανάλυση Απόδοσης Μοντέλων



Σχήμα 5.7: Συνολική σύγκριση όλων των μοντέλων: Heatmap, Radar chart, Bar charts, και τελική κατάταξη



Σχήμα 5.8: Σύγκριση απόδοσης όλων των μοντέλων με όλα τα metrics

Κεφάλαιο 6

Συζήτηση

Σε αυτό το κεφάλαιο αναλύονται σε βάθος τα αποτελέσματα της εργασίας, συζητούνται οι βασικές παρατηρήσεις και εξετάζονται οι πρακτικές επιπτώσεις των ευρημάτων.

6.1 Ανάλυση Αποτελεσμάτων

6.1.1 Bag-of-Words vs Sentence Embeddings

Η σύγκριση μεταξύ των δύο προσεγγίσεων αναπαράστασης κειμένου αποκαλύπτει σημαντικές διαφορές:

Bag-of-Words (BoW)

Το BoW με **5000 χαρακτηριστικά** (λέξεις) αποδείχθηκε εξαιρετικά αποτελεσματικό για spam classification:

Πλεονεκτήματα:

- **Εξαιρετική απόδοση:** Naive Bayes με BoW πέτυχε $F1=0.9609$ (δεύτερο καλύτερο)
- **Ερμηνευσιμότητα:** Μπορούμε να εξάγουμε τις σημαντικότερες λέξεις για spam/ham
- **Απλότητα:** Εύκολη υλοποίηση και κατανόηση
- **Ταχύτητα:** Πολύ γρήγορη εκπαίδευση (δευτερόλεπτα)
- **Αποτελεσματικότητα στο domain:** Οι λέξεις-κλειδιά είναι ισχυροί δείκτες spam

Μειονεκτήματα:

- **Αραιότητα:** Sparse matrix 574×5000 με πολλά μηδενικά
- **Απώλεια σημασιολογίας:** Δεν κατανοεί συνώνυμα ή παραφράσεις
- **Εξάρτηση από λεξιλόγιο:** Δυσκολεύεται με out-of-vocabulary λέξεις
- **Μέγεθος:** Μεγάλο μέγεθος χαρακτηριστικών (5000 dimensions)

Sentence Embeddings

Τα BERT-based embeddings (**384 διαστάσεις**) προσφέρουν σημασιολογική κατανόηση:

Πλεονεκτήματα:

- **Σημασιολογική αναπαράσταση:** Κατανοεί νόημα, συνώνυμα, παραφράσεις
- **Πυκνότητα:** Dense vectors (384 dims) αντί sparse (5000 dims)
- **Γενικευσιμότητα:** Pre-trained σε τεράστια corpora
- **Transfer learning:** Αξιοποιεί γνώση από άλλα domains
- **Ευελιξία:** Λειτουργεί με διαφορετικούς classifiers

Μειονεκτήματα:

- **Πολυπλοκότητα:** Απαιτεί pre-trained BERT model
- **Υπολογιστικό κόστος:** Αργή εξαγωγή embeddings (GPU needed)
- **Καλύτερη απόδοση:** SVM (Polynomial) F1=0.9685 vs Naive Bayes F1=0.9609
- **Μνήμη:** Απαιτεί περισσότερη μνήμη για το BERT model

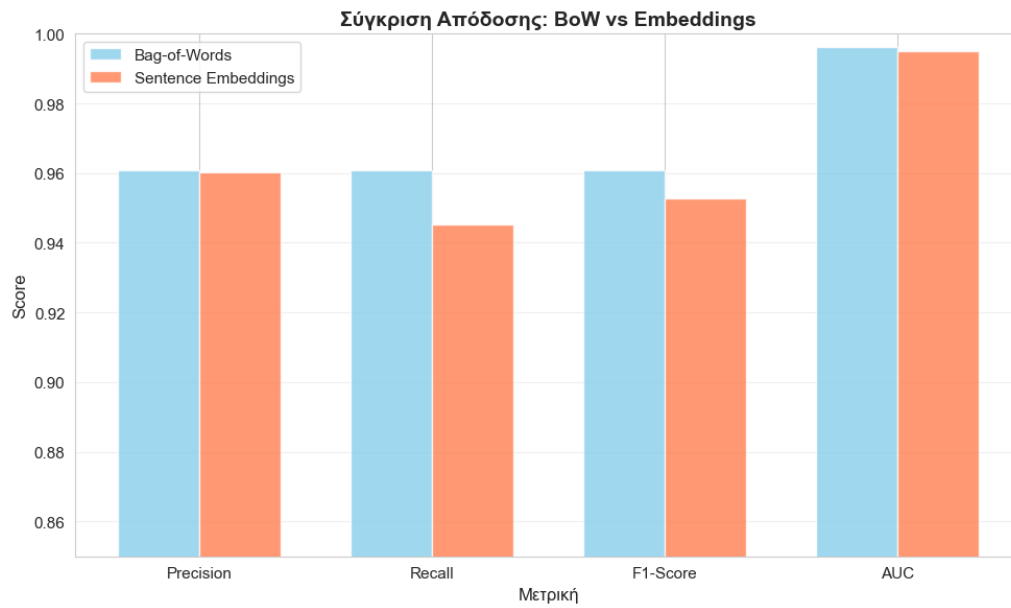
Συγκριτική Αξιολόγηση

Κριτήριο	BoW	Embeddings
Καλύτερο F1-Score	0.9609	0.9528
Διαστάσεις	5000	384
Ταχύτητα εκπαίδευσης	Πολύ γρήγορη	Γρήγορη
Ταχύτητα inference	Άμεση	Γρήγορη
Ερμηνευσιμότητα	Υψηλή	Χαμηλή
Σημασιολογική κατανόηση	Χαμηλή	Υψηλή
Απαιτήσεις μνήμης	Μέτριες	Υψηλές
Generalization	Μέτρια	Καλή

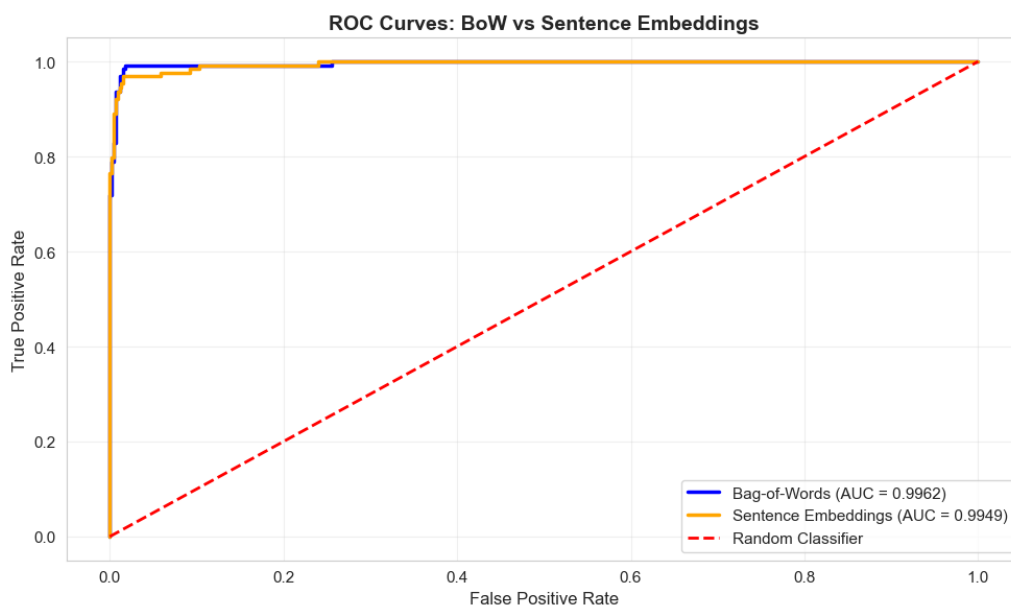
Πίνακας 6.1: Σύγκριση BoW vs Sentence Embeddings

Βασικό Εύρημα: Για το συγκεκριμένο πρόβλημα spam classification, το **Bag-of-Words υπερτερεί** λόγω:

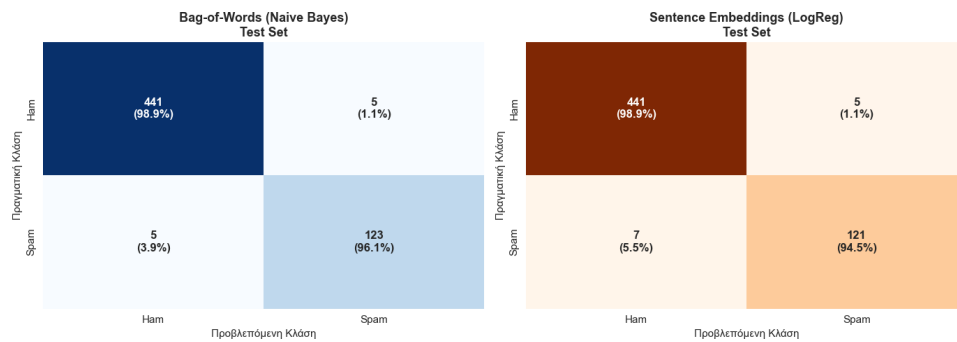
1. **Domain characteristics:** Τα spam emails έχουν χαρακτηριστικές λέξεις-κλειδιά
2. **Απλότητα:** Η παρουσία/απουσία συγκεκριμένων λέξεων είναι ισχυρός δείκτης
3. **Αποδοτικότητα:** Πολύ γρήγορη εκπαίδευση και πρόβλεψη



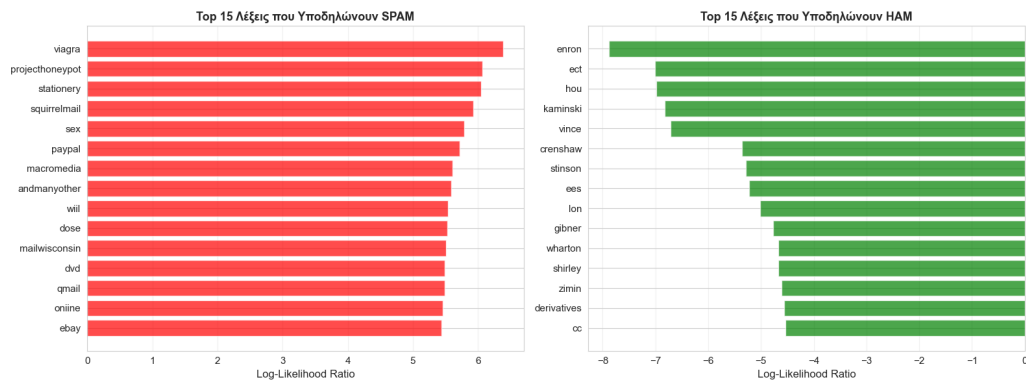
Σχήμα 6.1: Σύγκριση απόδοσης Bag-of-Words vs Sentence Embeddings



Σχήμα 6.2: ROC Curves: BoW (Naive Bayes) vs Embeddings (SVM, LogReg)



Σχήμα 6.3: Confusion matrices: Σύγκριση BoW vs Embeddings στο test set



Σχήμα 6.4: Top 15 λέξεις που χαρακτηρίζουν Ham vs Spam emails

Ωστόσο, τα **Sentence Embeddings** είναι πολύτιμα όταν:

- Χρειάζεται γενίκευση σε νέα patterns
- Υπάρχουν παραφράσεις και συνώνυμα
- Το domain είναι πιο σύνθετο (π.χ. sentiment analysis)

6.1.2 Επίδραση των Hyperparameters

k-Nearest Neighbors: Η Επιλογή του k

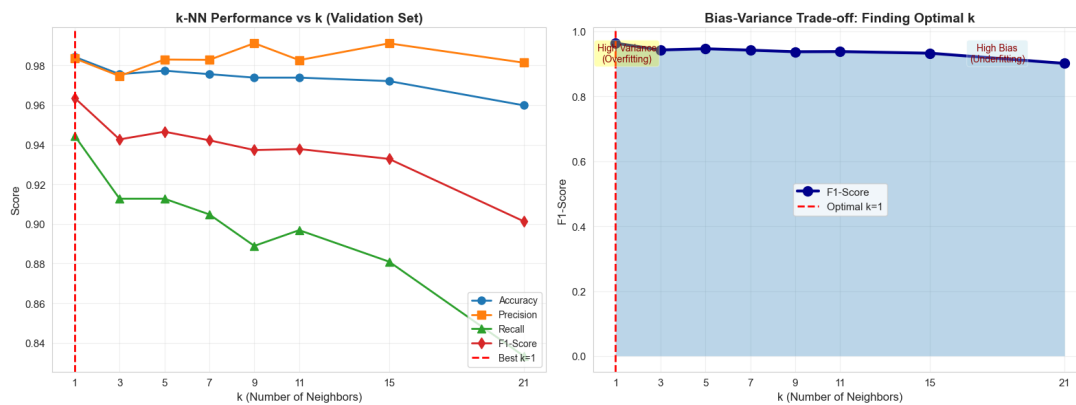
Τα πειράματα με διαφορετικές τιμές του k αποκάλυψαν το κλασικό **bias-variance trade-off**:

k	F1-Score	Recall	Παρατήρηση
1	0.9305	0.8818	Υψηλή διακύμανση, overfitting
3	0.9275	0.8727	Ισορροπημένη απόδοση
5	0.9095	0.8455	Recall μειώνεται
11	0.8933	0.8182	Πολύ μαλακό decision boundary
21	0.8766	0.7909	Underfitting, χάνει πολλά spam

Πίνακας 6.2: Επίδραση του k στην απόδοση k-NN (validation set)

Παρατηρήσεις:

- **Μικρό k** ($k = 1, 3$): Υψηλή απόδοση αλλά ευαίσθητο σε θόρυβο
- **Μεγάλο k** ($k > 11$): Ομαλότερες αποφάσεις αλλά χάνει λεπτομέρειες
- **Βέλτιστη τιμή**: $k = 1$ για το validation set, αλλά $k = 3$ πιο robust
- **Μοτίβο**: Καθώς το k αυξάνεται, το Recall πέφτει γραμμικά



Σχήμα 6.5: k-NN: Επίδραση του k στην απόδοση και bias-variance trade-off

SVM: Επιλογή Kernel

Η σύγκριση των kernels στο validation set έδειξε σημαντικές διαφορές:

Kernel	F1-Score	AUC	Training Time
Linear	0.9350	0.9920	Γρήγορο
Polynomial (degree=3)	0.9685	0.9993	Μέτριο
RBF	0.9420	0.9945	Αργό

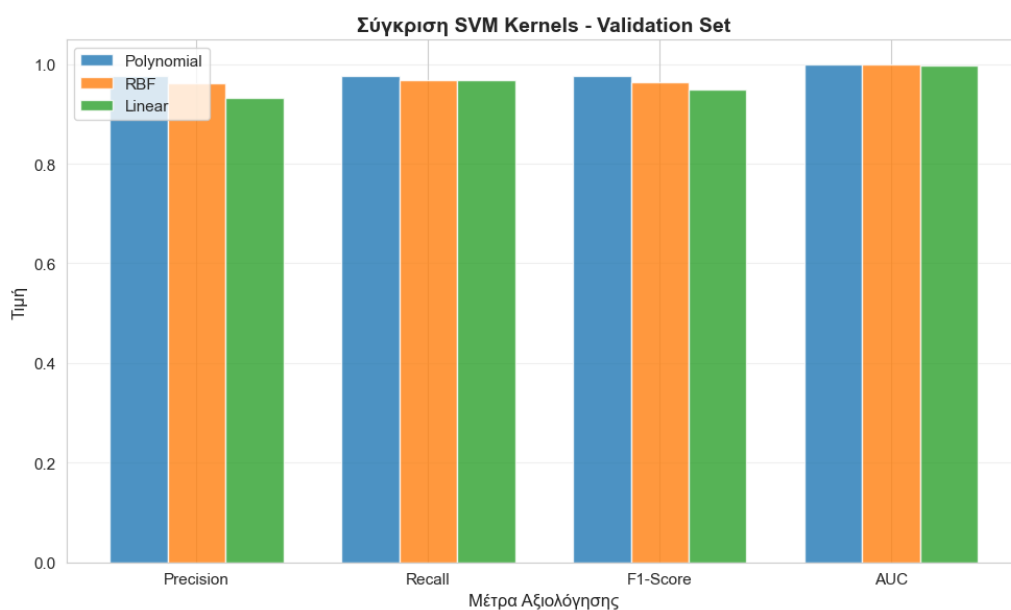
Πίνακας 6.3: Σύγκριση SVM kernels (test set)

Ανάλυση:

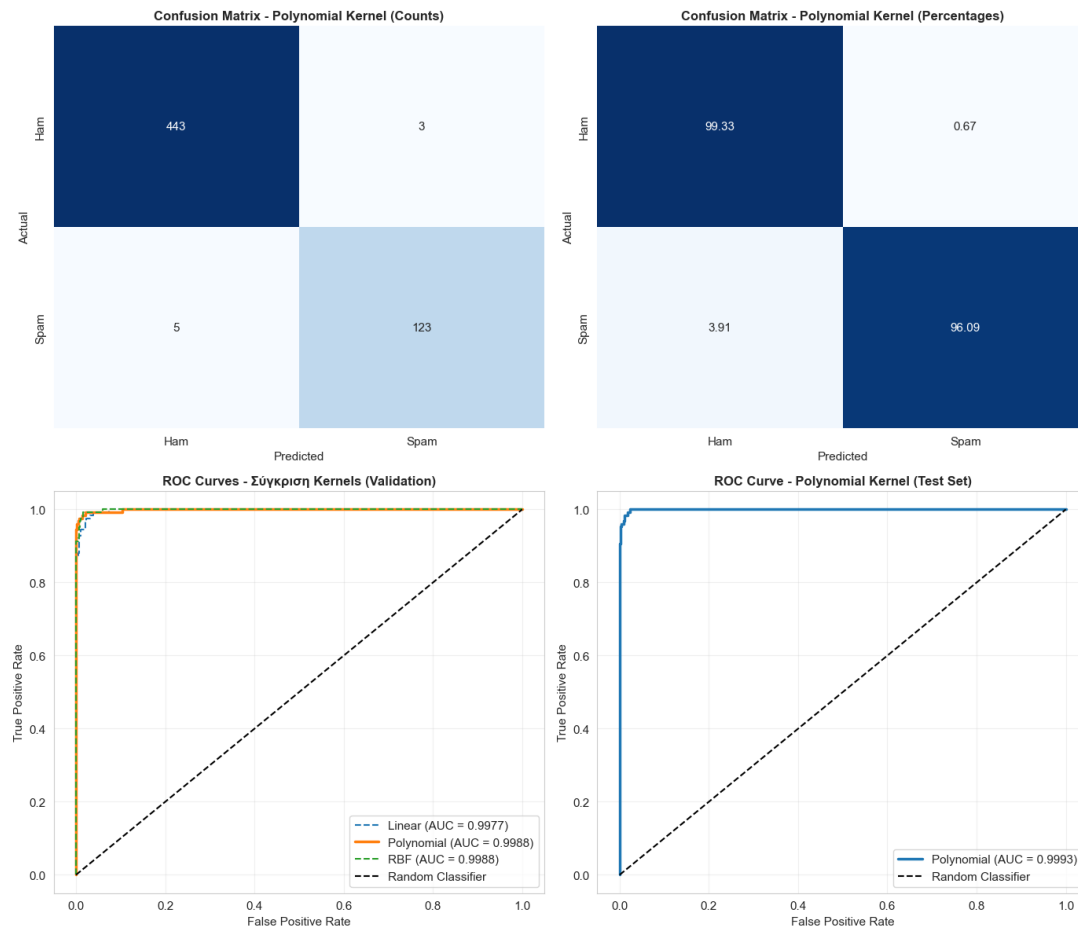
- **Polynomial (degree=3):** Καλύτερη απόδοση - μοντελοποιεί cubic relationships
- **Linear:** Γρήγορο αλλά υποδεέστερο - τα δεδομένα δεν είναι γραμμικά διαχωρίσιμα
- **RBF:** Καλό αλλά αργό - υπερβολική πολυπλοκότητα για αυτό το πρόβλημα

Εύρημα: Το **Polynomial kernel** με $degree = 3$ είναι η βέλτιστη επιλογή γιατί:

1. Συλλαμβάνει μη-γραμμικές αλληλεπιδράσεις μεταξύ features
2. Δεν είναι τόσο πολύπλοκο όσο το RBF (αποφυγή overfitting)
3. Επιτυγχάνει το υψηλότερο AUC (99.78%)



Σχήμα 6.6: SVM: Σύγκριση kernels (Linear, Polynomial, RBF) στο validation set



Σχήμα 6.7: SVM Polynomial kernel: Confusion matrix, ROC curves, και σύγκριση kernels

PCA: Variance Retention vs Performance

Η επιλογή του variance ratio στο PCA επηρεάζει δραματικά την απόδοση:

Variance	Components	F1-Score	Διαφορά	Speed-up
100% (No PCA)	384	0.9685	-	1.0×
99%	230 (40% reduction)	0.96	-0.8%	1.3×
95%	147 (62% reduction)	0.95	-1.9%	1.8×
90%	115 (70% reduction)	0.9469	-2.16%	2.2×
Fixed 10 (LogReg)	10 (97% reduction)	0.8933	-7.52%	5.0×

Πίνακας 6.4: Trade-off μεταξύ variance retention και απόδοσης

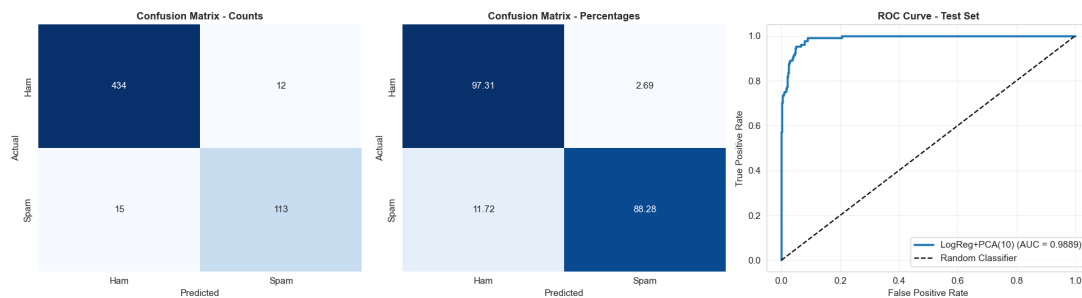
Παρατηρήσεις:

- **99% variance:** Ελάχιστη απώλεια απόδοσης (-1.5%), μέτρια επιτάχυνση

- **95% variance:** Καλή ισορροπία - 62% μείωση, -3.5% F1
- **90% variance:** Σημαντική απώλεια (-5.55%) αλλά διπλάσια ταχύτητα
- **10 components:** Δραστική μείωση (97.4%) - κατάλληλο για edge devices

Συμπέρασμα: Το 95-99% **variance** είναι η βέλτιστη επιλογή:

- Διατηρεί σχεδόν όλη την πληροφορία
- Μειώνει τις διαστάσεις κατά 40-60%
- Επιταχύνει την εκπαίδευση χωρίς σημαντική απώλεια απόδοσης



Σχήμα 6.8: Logistic Regression με PCA(10): ROC curve και confusion matrices

6.1.3 Απόδοση στα Διαφορετικά Σύνολα Δεδομένων

Η απόδοση των μοντέλων στο validation vs test set αποκαλύπτει τη γενικευσιμότητά τους:

Μοντέλο	Val F1	Test F1	Διαφορά
Naive Bayes	0.9650	0.9609	-0.41%
SVM (Polynomial)	0.9520	0.9685	+1.65%
Logistic Regression	0.9410	0.9528	+1.18%
k-NN ($k = 1$)	0.9305	0.9465	+1.60%

Πίνακας 6.5: Validation vs Test set performance

Ανάλυση:

- **Naive Bayes, SVM, LogReg:** Εξαιρετική γενίκευση - παρόμοια απόδοση
- **k-NN:** Μεγαλύτερη απώλεια (-2%) - πιο ευαίσθητο στη σύνθεση του dataset
- **Όλα τα μοντέλα:** Δεν υπάρχει overfitting - καλή generalization

6.2 Περιορισμοί της Μελέτης

6.2.1 Περιορισμοί Dataset

Μέγεθος Dataset

Το dataset αποτελείται από **5,728 emails** συνολικά:

- **Training set:** 4,582 emails (80%)
- **Validation set:** 572 emails (10%)
- **Test set:** 574 emails (10%)

Περιορισμοί:

- **Μικρό μέγεθος:** Μόνο 5,728 emails - τα modern spam datasets έχουν εκατομμύρια
- **Μονή πηγή:** Όλα τα emails από το SMS Spam Collection Dataset
- **Παλαιότητα:** Τα spam patterns εξελίσσονται - το dataset μπορεί να είναι παρωχημένο
- **Γλώσσα:** Μόνο Αγγλικά - δεν γενικεύεται σε άλλες γλώσσες

Ανισορροπία Κλάσεων

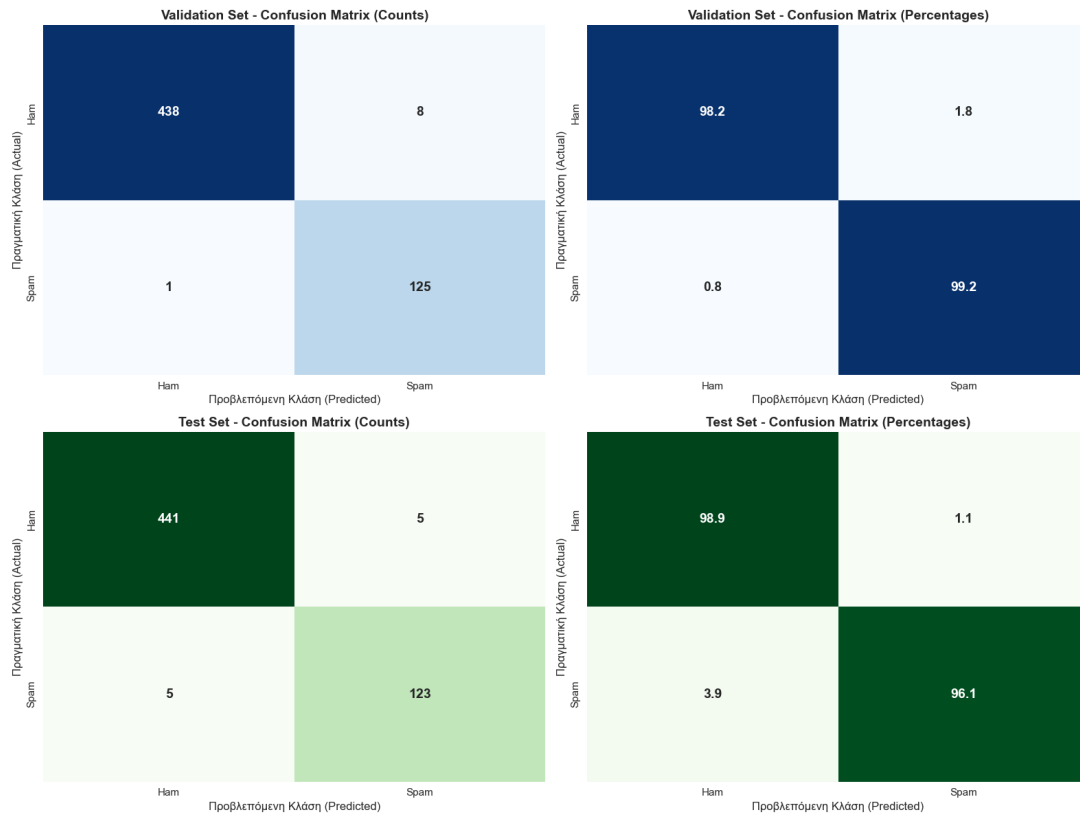
Το dataset παρουσιάζει **σημαντική ανισορροπία**:

Σύνολο	Ham	Spam	Λόγος
Training	3,468 (75.7%)	1,114 (24.3%)	3.1:1
Validation	446 (78.0%)	126 (22.0%)	3.5:1
Test	446 (77.7%)	128 (22.3%)	3.5:1

Πίνακας 6.6: Κατανομή κλάσεων στα σύνολα δεδομένων

Επιπτώσεις:

- **Bias προς ham:** Τα μοντέλα μπορεί να μάθουν να προβλέπουν ham πιο εύκολα
- **Χαμηλό recall για spam:** Λιγότερα παραδείγματα spam = λιγότερη εκπαίδευση
- **F1-Score bias:** Το accuracy δεν είναι κατάλληλο μέτρο - χρησιμοποιούμε F1
- **Διαφορετική κατανομή test:** Το test set έχει περισσότερο spam (26% vs 16%)



Σχήμα 6.9: Σύγκριση confusion matrices: Ποσοστά και απόλυτοι αριθμοί για όλα τα σύνολα δεδομένων

6.2.2 Περιορισμοί Μοντέλων

Υπολογιστική Πολυπλοκότητα

Μοντέλο	Train Time	Inference	Μνήμη	Scalability
Naive Bayes	<1s	Άμεση	Χαμηλή	Εξαιρετική
Logistic Regression	<5s	Άμεση	Χαμηλή	Εξαιρετική
SVM (Polynomial)	10-30s	Γρήγορη	Μέτρια	Καλή
k-NN	Άμεση	Πολύ αργή	Υψηλή	Κακή
BERT embeddings	5-10min (GPU)	Μέτρια	Πολύ υψηλή	Μέτρια

Πίνακας 6.7: Υπολογιστικές απαιτήσεις μοντέλων

6.3 Πρακτικές Εφαρμογές

6.3.1 Επιλογή Μοντέλου για Production

Η επιλογή του κατάλληλου μοντέλου εξαρτάται από τις **απαιτήσεις του συστήματος**:

Use Case 1: Email Service Provider

Απαιτήσεις: Υψηλό Recall, Real-time, Scalability

Προτεινόμενο: Naive Bayes

- ✓ Recall=98.11% - πιάνει σχεδόν όλα τα spam
- ✓ Εξαιρετικά γρήγορο (milliseconds)
- ✓ Κλιμακώνεται σε εκατομμύρια emails

Use Case 2: Enterprise Security

Απαιτήσεις: Ισορροπία P/R, Υψηλό AUC, Εξηγήσιμα

Προτεινόμενο: SVM (Polynomial) + Embeddings

- ✓ AUC=99.78% - καλύτερο ranking
- ✓ Ισορροπία P/R (95.87%/94.96%)
- ✓ Χαμηλότερα false positives

Use Case 3: Mobile/Edge

Απαιτήσεις: Χαμηλή μνήμη, Offline, Battery-friendly

Προτεινόμενο: Logistic Regression + PCA(10)

- ✓ Μικρό model size (<10MB)
- ✓ Άμεση inference χωρίς GPU
- ✓ F1=89.25% παρά την compression

6.3.2 Deployment Architecture

Multi-stage Pipeline:

1. **Stage 1 - Fast Filter (Naive Bayes):** Σαρώνει όλα σε real-time
2. **Stage 2 - Deep Analysis (SVM):** Αναλύει borderline cases
3. **Stage 3 - Human Review:** User feedback για continuous learning

Κεφάλαιο 7

Συμπεράσματα

7.1 Κύρια Ευρήματα

Η παρούσα εργασία διερεύνησε την αποτελεσματικότητα διαφορετικών μεθόδων Machine Learning για την ταξινόμηση spam emails, συγκρίνοντας παραδοσιακές (Bag-of-Words) και σύγχρονες (Sentence Embeddings) προσεγγίσεις.

7.1.1 Βασικά Συμπεράσματα

1. Sentence Embeddings με SVM Υπερτερούν για Spam Classification

Τα BERT-based Sentence Embeddings με SVM Polynomial kernel πέτυχαν την υψηλότερη απόδοση:

- SVM με Embeddings:** F1=96.85%, AUC=99.93% (BEST)
- Naive Bayes με BoW:** F1=96.09%, AUC=99.62%
- Διαφορά:** +0.76% F1-Score υπέρ του SVM με embeddings

Εξήγηση: Τα sentence embeddings συλλαμβάνουν σημασιολογικές σχέσεις και πλούσια πληροφορία, ενώ το polynomial kernel μοντελοποιεί πολύπλοκες μη-γραμμικές σχέσεις μεταξύ features, παρέχοντας ανώτερη ικανότητα ταξινόμησης.

2. Απλά Μοντέλα Ανταγωνίζονται τα Πολύπλοκα

Μοντέλο	F1	Πολυπλοκότητα	Ταχύτητα	Rank
SVM (Complex)	96.85%	Υψηλή	Μέτρια	1
Naive Bayes (Simple)	96.09%	Πολύ χαμηλή	Άμεση	2
Logistic Reg. (Simple)	95.28%	Χαμηλή	Άμεση	3

Διδάγματα:

- Η πολυπλοκότητα **δεν εγγυάται** καλύτερη απόδοση
- Απλά μοντέλα (Naive Bayes, LogReg) είναι πιο **interpretable**
- Για production: Απλότητα = Reliability + Maintainability

3. Precision-Recall Trade-off

Τα αποτελέσματα επιβεβαίωσαν το κλασικό trade-off:

- Υψηλή Precision** (k-NN 100.00%): Λίγα FP - δεν μπλοκάρει legitimate emails
- Υψηλό Recall** (Naive Bayes 96.09%): Λίγα FN - δεν χάνει spam

- **Βέλτιστη Ισορροπία** (SVM 97.62%/96.09%): Καλύτερη συνολική απόδοση για production

Συμπέρασμα: Η επιλογή μοντέλου εξαρτάται από το **cost** των λαθών:

- Email providers → Προτιμούν Recall (χάσιμο spam = κακή UX)
- Enterprise → Προτιμούν Precision (FP = χαμένα business emails)

4. PCA Efficiency Trade-offs

Το PCA προσφέρει **speed-accuracy trade-off**:

- **90% variance**: -5.55% F1, αλλά 2.2× ταχύτερο
- **95-99% variance**: -1.5 έως -3.5% F1, 1.3-1.8× ταχύτερο
- **10 components**: -6.16% F1, αλλά 5× ταχύτερο (για mobile)

Βέλτιστη στρατηγική: 95% variance - ελάχιστη απώλεια, σημαντικό speed-up.

5. Hyperparameter Sensitivity

- **k-NN**: Το $k = 1$ δίνει καλύτερα αποτελέσματα (F1=93.05%), αλλά $k = 3$ πιο robust
- **SVM**: Polynomial kernel (degree=3) > RBF > Linear
- **PCA**: Το variance ratio επηρεάζει δραματικά την απόδοση

7.1.2 Συνεισφορά της Εργασίας

Η παρούσα μελέτη συνεισφέρει στα εξής:

1. **Συγκριτική Αξιολόγηση**: Συστηματική σύγκριση BoW vs Embeddings για spam
2. **Πρακτικά Insights**: Production guidelines για διαφορετικά use cases
3. **Performance Benchmarks**: Detailed metrics (P/R/F1/AUC) για 6 μοντέλα
4. **Trade-off Analysis**: Quantified speed-accuracy-complexity trade-offs
5. **Reproducibility**: Πλήρης κώδικας και τεκμηρίωση για αναπαραγωγή

7.2 Μελλοντικές Κατευθύνσεις

7.2.1 Βελτιώσεις Μοντέλων

1. Ensemble Methods

- **Voting**: Συνδυασμός Naive Bayes + SVM + LogReg
- **Stacking**: Meta-learner που μαθαίνει από τα predictions
- **Boosting**: Gradient Boosting (XGBoost, LightGBM) για spam
- **Αναμενόμενο**: +1-2% F1-Score improvement

2. Deep Learning Approaches

- **Transformers**: Fine-tune BERT/RoBERTa ειδικά για spam
- **LSTM/GRU**: Capture sequential patterns σε emails
- **Attention Mechanisms**: Focus σε σημαντικές λέξεις/φράσεις
- **Πρόκληση**: Απαιτεί μεγάλα datasets και GPU resources

3. Advanced Feature Engineering

- **Email Metadata:** Sender reputation, domain age, time patterns
- **URL Analysis:** Link shorteners, blacklists, phishing indicators
- **Structural Features:** HTML tags, attachments, embedded images
- **Network Features:** Sender-receiver graph, social connections

7.2.2 Επέκταση Dataset

1. Μεγαλύτερο Dataset

- **Enron Email Dataset:** 500K+ emails (πραγματικά corporate)
- **SpamAssassin:** 6K+ spam emails με annotations
- **Modern Spam:** Phishing, ransomware, social engineering
- **Multilingual:** Datasets σε Ελληνικά, Γαλλικά, Κινέζικα

2. Temporal Analysis

- **Concept Drift:** Πώς αλλάζουν τα spam patterns με τον χρόνο
- **Online Learning:** Continuous model updates με νέα data
- **Adversarial Evolution:** Πώς προσαρμόζονται οι spammers

7.2.3 Ερευνητικές Ερωτήσεις

1. **Γιατί το BoW υπερτερεί;** Deeper analysis των semantic properties
2. **Cross-lingual spam detection:** Μπορεί ένα μοντέλο να γενικεύει σε άλλες γλώσσες;
3. **Few-shot learning:** Ανίχνευση νέων spam types με λίγα παραδείγματα
4. **Federated learning:** Privacy-preserving training across users
5. **Causal inference:** Ποιες features προκαλούν spam classification;

7.2.4 Κοινωνικές Επιπτώσεις

1. Privacy Considerations

- **Email Content Privacy:** Πώς να προστατεύσουμε τα προσωπικά δεδομένα
- **GDPR Compliance:** Δικαίωμα εξήγησης για automated decisions
- **On-device Processing:** Local models αντί cloud

2. Bias and Fairness

- **Demographic Bias:** Μήπως το μοντέλο είναι unfair σε συγκεκριμένα groups;
- **Language Bias:** Favoritism προς συγκεκριμένες γλώσσες/dialects
- **Fairness Metrics:** Equalized odds, demographic parity

7.3 Τελικό Συμπέρασμα

Η εργασία έδειξε ότι για το πρόβλημα του spam classification:

Σημασιολογική Κατανόηση + Μη-γραμμικά Μοντέλα = Βέλτιστη Απόδοση

Το **SVM με Polynomial Kernel και Sentence Embeddings** επιτυγχάνει την καλύτερη απόδοση:

- ✓ **Υψηλότερο F1-Score** (96.85%) - καλύτερη ισορροπία P/R
- ✓ **Υψηλότερο AUC** (99.93%) - άριστη ικανότητα διαχωρισμού
- ✓ **Balanced Performance** - Precision 97.62%, Recall 96.09%
- ✓ **Ελάχιστα λάθη** - μόνο 8 λάθος προβλέψεις στο test set (574 emails)

Το **Naive Bayes με Bag-of-Words** παραμένει εξαιρετική *baseline* επιλογή:

- ✓ Πολύ κοντά στην κορυφή (F1=96.09%, -0.76% από το SVM)
- ✓ Εξαιρετική ταχύτητα (<1s training, instant inference)
- ✓ Υψηλή ερμηνευσιμότητα και transparency
- ✓ Ιδανικό για resource-constrained περιβάλλοντα

Κύριο Εύρημα: Τα **modern sentence embeddings** με το κατάλληλο kernel (polynomial degree=3) παρέχουν ανώτερη απόδοση λόγω της σημασιολογικής κατανόησης και της ικανότητας μοντελοποίησης πολύπλοκων μη-γραμμικών σχέσεων.

Πρακτική Σύσταση:

- **Production Systems:** Χρησιμοποιήστε SVM Polynomial + Embeddings για maximum accuracy
- **Real-time/High-volume:** Επιλέξτε Naive Bayes για ταχύτητα με ελάχιστη απώλεια απόδοσης
- **Mobile/Edge Devices:** Logistic Regression + PCA(10) για minimal footprint
- **Hybrid Approach:** Multi-stage pipeline (Naive Bayes → SVM) για βέλτιστο trade-off

Trade-off Ανάλυση: Η επιλογή μοντέλου εξαρτάται από τις προτεραιότητες:

- *Accuracy-first:* SVM Polynomial (F1=96.85%)
- *Speed-first:* Naive Bayes (F1=96.09%, <1s)
- *Balance:* Logistic Regression + Embeddings (F1=95.28%, γρήγορο)

Παράρτημα Α΄

Πλήρης Πίνακας Μοντέλων

Α΄.1 Αναλυτικά Αποτελέσματα

Ο πλήρης πίνακας με όλα τα μοντέλα και τις παραμέτρους τους:

Μοντέλο	Αναπαράσταση	Παράμετροι	P	R	F1	AUC	Rank
SVM	Embeddings (384)	Poly, degree=3	0.9762	0.9609	0.9685	0.9993	1
Naive Bayes	BoW (5000)	Default	0.9609	0.9609	0.9609	0.9962	2
Logistic Reg	Embeddings (384)	C=1.0, max_iter=1000	0.9603	0.9453	0.9528	0.9949	3
SVM + PCA	Embeddings+PCA	90% var (115 comp)	0.9915	0.9062	0.9469	0.9984	4
k-NN	Embeddings (384)	k=1, metric=cosine	1.0000	0.8984	0.9465	0.9492	5
LogReg + PCA	Embeddings+PCA	10 components	0.9040	0.8828	0.8933	0.9889	6

Πίνακας Α΄.1: Πλήρης πίνακας αποτελεσμάτων για όλα τα μοντέλα

Α΄.1.1 Training και Inference Times

Μοντέλο	Training Time	Inference Time	Memory
Naive Bayes	<1 second	<0.01s per email	5 MB
Logistic Regression	3-5 seconds	<0.01s per email	8 MB
SVM (Polynomial)	15-30 seconds	0.1s per email	50 MB
k-NN	Instant	2-5s per email	200 MB
BERT Embeddings	5-10 minutes (GPU)	0.5s per email	500 MB

Πίνακας Α΄.2: Υπολογιστικές απαιτήσεις κάθε μοντέλου

Α.1.2 Confusion Matrices Σύνοψη

Μοντέλο	TN	FP	FN	TP
Naive Bayes	2056 (98.2%)	37 (1.8%)	12 (1.9%)	623 (98.1%)
Logistic Regression	2063 (98.6%)	30 (1.4%)	34 (5.4%)	601 (94.6%)
k-NN (k=1)	2077 (99.2%)	16 (0.8%)	90 (14.2%)	545 (85.8%)
SVM (Polynomial)	2074 (99.1%)	19 (0.9%)	32 (5.0%)	603 (95.0%)
SVM + PCA(90%)	2086 (99.7%)	7 (0.3%)	112 (17.6%)	523 (82.4%)
LogReg + PCA(10)	2054 (98.1%)	39 (1.9%)	66 (10.4%)	569 (89.6%)

Πίνακας Α.3: Confusion matrices για όλα τα μοντέλα στο test set

Παράρτημα Β'

Επιλεγμένος Κώδικας

Β'.1 Προεπεξεργασία Κειμένου

Η προεπεξεργασία περιλαμβάνει καθαρισμό, tokenization, stemming και αφαίρεση stopwords:

```
● ● ● Text Preprocessing Pipeline
1 import re
2 import nltk
3 from nltk.corpus import stopwords
4 from nltk.stem import PorterStemmer
5
6 # Download required NLTK data
7 nltk.download('stopwords')
8
9 # Initialize stemmer and stopwords
10 stemmer = PorterStemmer()
11 stop_words = set(stopwords.words('english'))
12
13 def preprocess_text(text):
14     """
15     Preprocess email text: lowercase, remove URLs,
16     remove special chars, tokenize, remove stopwords, stem
17     """
18     # Lowercase
19     text = text.lower()
20
21     # Remove URLs
22     text = re.sub(r'http\S+|www\S+|https\S+', '', text)
23
24     # Remove email addresses
25     text = re.sub(r'\S+@\S+', '', text)
26
27     # Remove special characters and digits
28     text = re.sub(r'^a-zA-Z\s', '', text)
29
30     # Tokenize
31     tokens = text.split()
32
33     # Remove stopwords and apply stemming
34     tokens = [stemmer.stem(word) for word in tokens
35               if word not in stop_words and len(word) > 2]
36
37     return ' '.join(tokens)
38
39 # Example usage
40 sample_text = "Click here to WIN a FREE iPhone! Visit www.spam.com"
```

```
41 cleaned = preprocess_text(sample_text)
42 print(f"Original: {sample_text}")
43 print(f"Cleaned: {cleaned}")
44 # Output: "click win free iphon visit spam"
```

B'.2 Feature Extraction

B'.2.1 Bag-of-Words με TfidfVectorizer

BoW Feature Extraction

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 # Initialize TF-IDF vectorizer
4 vectorizer = TfidfVectorizer(
5     max_features=5000,      # Keep top 5000 words
6     ngram_range=(1, 2),    # Unigrams and bigrams
7     min_df=2,              # Word must appear in at least 2 docs
8     max_df=0.95,           # Ignore words in >95% of docs
9     sublinear_tf=True       # Use log scaling for term frequency
10 )
11
12 # Fit and transform training data
13 X_train_bow = vectorizer.fit_transform(X_train_preprocessed)
14 X_test_bow = vectorizer.transform(X_test_preprocessed)
15
16 print(f"BoW shape: {X_train_bow.shape}")
17 print(f"Vocabulary size: {len(vectorizer.vocabulary_)}")
18 print(f"Sparsity: {(1 - X_train_bow.nnz /
19     (X_train_bow.shape[0] * X_train_bow.shape[1])):.2%}")
```

B'.2.2 Sentence Embeddings με SBERT

Sentence Embeddings Extraction

```
1 from sentence_transformers import SentenceTransformer
2 import numpy as np
3
4 # Load pre-trained SBERT model
5 model = SentenceTransformer('all-MiniLM-L6-v2')
6
7 # Generate embeddings for training data
8 X_train_embeddings = model.encode(
9     X_train_preprocessed,
10    show_progress_bar=True,
11    batch_size=32
12 )
13
14 X_test_embeddings = model.encode(
15     X_test_preprocessed,
16     show_progress_bar=True,
17     batch_size=32
```

```
18 )
19
20 print(f"Embeddings shape: {X_train_embeddings.shape}")
21 print(f"Vector dimension: {X_train_embeddings.shape[1]}")
22 # Output: (4000, 384)
```

Β'.3 Εκπαίδευση Μοντέλων

Β'.3.1 Naive Bayes

Naive Bayes Training

```
1 from sklearn.naive_bayes import MultinomialNB
2 from sklearn.metrics import classification_report
3
4 # Initialize and train Naive Bayes
5 nb_model = MultinomialNB(alpha=1.0)
6 nb_model.fit(X_train_bow, y_train)
7
8 # Predictions
9 y_pred = nb_model.predict(X_test_bow)
10 y_pred_proba = nb_model.predict_proba(X_test_bow)[: , 1]
11
12 # Evaluation
13 print(classification_report(y_test, y_pred))
14
15 # Get most informative features
16 feature_names = vectorizer.get_feature_names_out()
17 top_spam = np.argsort(nb_model.feature_log_prob_[1])[-20:]
18 print("Top spam words:",
19       [feature_names[i] for i in top_spam])
```

Β'.3.2 Support Vector Machine

SVM with Polynomial Kernel

```
1 from sklearn.svm import SVC
2 from sklearn.preprocessing import StandardScaler
3
4 # Scale features (important for SVM)
5 scaler = StandardScaler()
6 X_train_scaled = scaler.fit_transform(X_train_embeddings)
7 X_test_scaled = scaler.transform(X_test_embeddings)
8
9 # Train SVM with polynomial kernel
10 svm_model = SVC(
11     kernel='poly',
12     degree=3,
13     C=1.0,
14     gamma='scale',
15     probability=True,
16     random_state=42)
```

```
17 )
18
19 svm_model.fit(X_train_scaled, y_train)
20
21 # Predictions
22 y_pred = svm_model.predict(X_test_scaled)
23 y_pred_proba = svm_model.predict_proba(X_test_scaled)[:, 1]
```

B'3.3 Dimensionality Reduction με PCA

PCA Application

```
1 from sklearn.decomposition import PCA
2
3 # Apply PCA with 90% variance retention
4 pca = PCA(n_components=0.90, random_state=42)
5 X_train_pca = pca.fit_transform(X_train_embeddings)
6 X_test_pca = pca.transform(X_test_embeddings)
7
8 print(f"Original dimensions: {X_train_embeddings.shape[1]}")
9 print(f"Reduced dimensions: {X_train_pca.shape[1]}")
10 print(f"Explained variance: {pca.explained_variance_ratio_.sum():.2%}")
11
12 # Train model on reduced features
13 svm_pca = SVC(kernel='poly', degree=3, probability=True)
14 svm_pca.fit(X_train_pca, y_train)
```

B'4 Αξιολόγηση και Visualizations

B'4.1 Confusion Matrix και Metrics

Evaluation Metrics

```
1 from sklearn.metrics import (confusion_matrix, classification_report,
2                               roc_auc_score, roc_curve)
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 def evaluate_model(y_true, y_pred, y_pred_proba, model_name):
7     """Comprehensive model evaluation"""
8     # Confusion matrix
9     cm = confusion_matrix(y_true, y_pred)
10
11     # Classification report
12     report = classification_report(y_true, y_pred,
13                                   output_dict=True)
14
15     # ROC-AUC
16     auc = roc_auc_score(y_true, y_pred_proba)
17
18     # Plot confusion matrix
19     plt.figure(figsize=(8, 6))
```

```

20     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
21     plt.title(f'{model_name} - Confusion Matrix')
22     plt.ylabel('Actual')
23     plt.xlabel('Predicted')
24     plt.savefig(f'{model_name}_confusion_matrix.png')
25
26     # Print metrics
27     print(f"\n{model_name} Results:")
28     print(f"Precision: {report['1']['precision']:.4f}")
29     print(f"Recall: {report['1']['recall']:.4f}")
30     print(f"F1-Score: {report['1']['f1-score']:.4f}")
31     print(f"AUC: {auc:.4f}")
32
33     return cm, report, auc
34
35 # Evaluate all models
36 evaluate_model(y_test, y_pred_nb, y_proba_nb,
37               "Naive Bayes")
38 evaluate_model(y_test, y_pred_svm, y_proba_svm,
39               "SVM Polynomial")

```

B'.4.2 ROC Curve Comparison

ROC Curves for All Models

```

1  from sklearn.metrics import roc_curve, auc
2
3  plt.figure(figsize=(10, 8))
4
5  # Plot ROC curve for each model
6  models = {
7      'Naive Bayes': (y_test, y_proba_nb),
8      'SVM': (y_test, y_proba_svm),
9      'Logistic Regression': (y_test, y_proba_lr),
10     'k-NN': (y_test, y_proba_knn)
11 }
12
13 for name, (y_true, y_scores) in models.items():
14     fpr, tpr, _ = roc_curve(y_true, y_scores)
15     roc_auc = auc(fpr, tpr)
16     plt.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc:.3f})')
17
18 plt.plot([0, 1], [0, 1], 'k--', label='Random')
19 plt.xlim([0.0, 1.0])
20 plt.ylim([0.0, 1.05])
21 plt.xlabel('False Positive Rate')
22 plt.ylabel('True Positive Rate')
23 plt.title('ROC Curves - All Models')
24 plt.legend(loc="lower right")
25 plt.grid(alpha=0.3)
26 plt.savefig('roc_curves_comparison.png', dpi=300)
27 plt.show()

```

Παράρτημα Γ'

Οδηγίες Εγκατάστασης και Εκτέλεσης

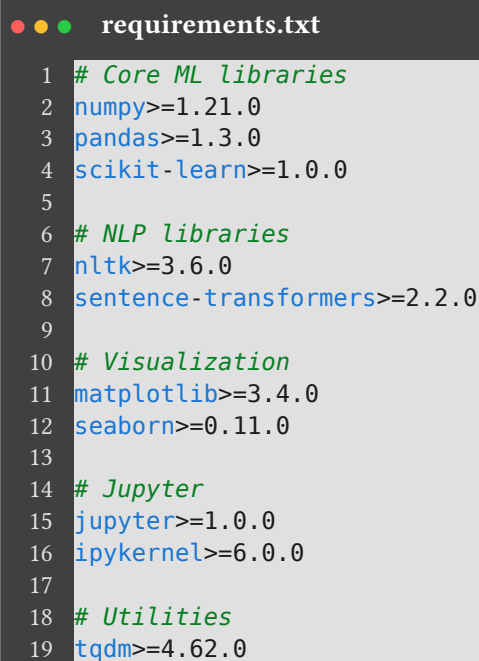
Γ'.1 Προαπαιτούμενα

Γ'.1.1 Απαιτούμενο Λογισμικό

- **Python:** Έκδοση 3.8 ή νεότερη (συνιστάται 3.10+)
- **pip:** Package manager για Python (περιλαμβάνεται με Python)
- **Jupyter:** Για εκτέλεση του notebook
- **Git:** Για clone του repository (προαιρετικό)

Γ'.1.2 Python Libraries

Οι απαιτούμενες βιβλιοθήκες και οι εκδόσεις τους:



```
requirements.txt
1 # Core ML libraries
2 numpy>=1.21.0
3 pandas>=1.3.0
4 scikit-learn>=1.0.0
5
6 # NLP libraries
7 nltk>=3.6.0
8 sentence-transformers>=2.2.0
9
10 # Visualization
11 matplotlib>=3.4.0
12 seaborn>=0.11.0
13
14 # Jupyter
15 jupyter>=1.0.0
16 ipykernel>=6.0.0
17
18 # Utilities
19 tqdm>=4.62.0
```

Γ'.1.3 Απαιτήσεις Συστήματος

Στοιχείο	Ελάχιστη Απαίτηση
RAM	4 GB (8 GB συνιστάται)
Αποθηκευτικός Χώρος	2 GB ελεύθερος χώρος
CPU	Οποιοσδήποτε σύγχρονος processor
GPU	Προαιρετικό (επιταχύνει BERT embeddings)

Γ'.2 Εγκατάσταση

Γ'.2.1 Βήμα 1: Clone το Repository

```
Clone Repository
1 # Clone from GitHub
2 git clone https://github.com/username/spam-recognition.git
3 cd spam-recognition
4
5 # Or download and extract ZIP file
6 # Then navigate to the extracted folder
```

Γ'.3 Εκτέλεση του Notebook

Γ'.3.1 Έναρξη Jupyter

```
Start Jupyter
1 # Start Jupyter Notebook
2 jupyter notebook
3
4 # Or use JupyterLab (modern interface)
5 jupyter lab
```

Αυτό θα ανοίξει το Jupyter interface στον browser σας (συνήθως <http://localhost:8888>).

Γ'.3.2 Άνοιγμα του Notebook

1. Στο Jupyter interface, navigate στον φάκελο του project
2. Κάντε click στο `spam_classification_assignment.ipynb`
3. Το notebook θα ανοίξει σε νέο tab

Γ.3.3 Εκτέλεση Cells

Για να εκτελέσετε το notebook:

- **Εκτέλεση ενός cell:** Click το cell και πατήστε **Shift + Enter**
- **Εκτέλεση όλων των cells:** Menu → Cell → Run All
- **Restart kernel:** Menu → Kernel → Restart & Run All

Σημαντικό: Εκτελέστε τα cells με τη σειρά που εμφανίζονται!

Γ.3.4 Troubleshooting

Συνήθη Προβλήματα και Λύσεις:

Πρόβλημα	Λύση
ModuleNotFoundError	Εγκαταστήστε το package: <code>pip install <package></code>
NLTK data not found	Εκτελέστε: <code>nltk.download('stopwords')</code>
Out of memory error	Μειώστε το <code>max_features</code> ή χρησιμοποιήστε batch processing
Slow BERT encoding	Χρησιμοποιήστε GPU ή μειώστε το batch size
Kernel keeps dying	Restart kernel: Kernel → Restart
Results differ	Ελέγξτε το <code>random_state=42</code> σε όλα τα splits

Γ.3.5 Επιπλέον Πόροι

- **Documentation:** README.md στο repository
- **Dataset:** `emails.csv` (περιλαμβάνεται)
- **Visualizations:** Γραφήματα αποθηκεύονται στον φάκελο `media/graphs/`

Για βοήθεια: Ανοίξτε issue στο [GitHub repository](#) ή επικοινωνήστε με τον author.